

PPO-Driven Swarm Control --core pipeline

End-to-End Hybrid Multi-Robot Coverage Pipeline with Local Perception, Potential-Field Geometry, Graph Coordination, and Role-Adaptive Swarm Behavior

1. Abstract

This notebook develops a hybrid multi-robot coverage pipeline for vegetation-aware exploration over satellite imagery. The system begins with a user-provided RGB satellite image and transforms it into a normalized scalar utility field using the Visible Atmospherically Resistant Index (VARI), an RGB-only vegetation proxy used when near-infrared information is unavailable. This utility field becomes the common substrate for perception, reward, coverage, and swarm-level evaluation.

A single-agent Proximal Policy Optimization (PPO) controller is trained on local 128×128 utility-field crops using a first-visit reward. The learned policy is not treated as a complete swarm controller. Instead, it is interpreted as a microscopic local navigation primitive: a learned directional instinct that can identify locally useful motion from partial observations.

To lift this learned local policy into a multi-agent system, the notebook adds classical multi-robot systems structure through three major layers:

1. **Artificial potential fields**, which provide utility-gradient attraction, inter-agent repulsion, revisit avoidance, and boundary shaping.
2. **Graph-based coordination**, which constructs a time-varying proximity graph and uses local consensus-like interaction to smooth neighbor behavior without collapsing the swarm into a rendezvous point.
3. **CRN-inspired stochastic role switching**, which assigns each agent a time-varying internal role and modulates its controller weights accordingly.

The final controller is therefore a role-conditioned hybrid policy:

$$u_i(k) = w_{\text{ppo}}(i, k) u_i^{\text{ppo}}(k) + w_{\text{pf}}(i, k) u_i^{\text{pf}}(k) + w_{\text{cons}}(i, k) u_i^{\text{cons}}(k)$$

The main claim is deliberately precise:

PPO learns local exploration instinct; classical multi-robot systems theory imposes collective structure; the contribution is the hybridization.

The final outputs include utility-field visualizations, PPO training diagnostics, single-agent evaluation, multi-agent rollouts, hybrid swarm trajectories, role-colored trajectory segments, communication-graph overlays, visit heatmaps, coverage metrics, role-population dynamics, and a final animation where local perception, potential-field shaping, graph interaction, and current agent roles are visually legible.

2. Core Idea

The central idea is to combine learned local autonomy with structured swarm coordination.

A pure reinforcement-learning controller can learn useful behavior from local image crops, but directly copying the same PPO policy across many agents is not enough for robust swarm behavior. Naive policy replication can produce clustering, redundant coverage, overlapping trajectories, weak coordination, and visually unconvincing collective motion.

Classical multi-robot systems theory addresses exactly these weaknesses. Potential fields can enforce spacing and geometric shaping. Graph models can express local communication and neighbor agreement. Stochastic task-switching can create heterogeneous swarm behavior from otherwise identical agents. Coverage metrics can quantify whether the swarm is actually exploring efficiently.

This notebook therefore builds a layered system with each layer having a clear purpose:

- PPO provides local vegetation-aware motion.
- Artificial potential fields provide geometric shaping.
- Graph coordination provides local swarm coherence.
- Role switching provides adaptive heterogeneity.
- Metrics and visualizations verify whether the behavior is actually meaningful.

This is not a vague “RL for swarms” demonstration. It is a hybrid control pipeline where reinforcement learning and classical multi-robot systems theory play separate but complementary roles.

3. Conceptual Positioning

This notebook is positioned as a decentralized multi-agent coverage system over a scalar utility field.

The environment is represented by a utility map

$$\phi(x, y) \in [0, 1]$$

where high values correspond to vegetation-rich or informative regions. Each agent only observes a local crop of this field. The PPO policy therefore does not receive a global map or a complete planning problem. It receives partial local evidence and proposes a local action.

The swarm layer then organizes many such local agents using multi-robot coordination mechanisms:

- local sensing through 128×128 crops,
- local communication through a proximity graph,
- local geometric shaping through artificial potential fields,
- local stochastic role transitions through context-dependent role switching,
- global evaluation through offline metrics such as coverage, redundancy, utility gain, spacing, and consensus error.

The scientific structure is:

microscopic learned policy+macroscopic swarm structure = interpretable hybrid coverage beh

The system is designed for a precision-agriculture-style narrative, but the method is general enough to represent other scalar-field inspection problems such as environmental monitoring, distributed search, mapping-inspired exploration, or adaptive inspection over utility landscapes.

4. Theoretical Anchors

This pipeline follows the main mathematical language of multi-robot systems:

4.1 Decentralized swarm control

Each agent acts from local information. The policy does not require centralized global planning during execution. The simulation can still

maintain global bookkeeping for evaluation, reward calculation, plotting, and diagnostics, but the controller itself is structured around local observation and local interaction.

4.2 Single-integrator agent dynamics

Each robot is modeled as a 2D point agent with discrete-time single-integrator dynamics:

$$p_i(k+1) = p_i(k) + \Delta t u_i(k),$$

where

$$p_i(k) \in \mathbb{R}^2$$

is the position of agent i , and

$$u_i(k) \in \mathbb{R}^2$$

is the velocity-like control input.

This abstraction is appropriate because the notebook focuses on high-level coverage behavior rather than low-level UAV attitude dynamics.

4.3 Potential-field control

Potential-field methods are used as shaping terms. They are not treated as globally complete planners. Their job is to bend motion using local geometry:

- attraction toward high-utility regions,
- repulsion from nearby agents,
- repulsion from heavily visited cells,
- soft repulsion from boundaries.

4.4 Graph-based consensus

A time-varying communication graph is built from inter-agent distance. Agents within a communication radius become neighbors. The graph allows local coordination through adjacency, degree, and Laplacian structure.

The goal is not to force all agents to meet at one point. Instead, the consensus layer acts as a local coordination smoother, reducing sharp disagreement between neighboring preferred directions while preserving coverage behavior.

4.5 CRN-inspired stochastic role switching

Each agent has an internal role:

$$\text{role}_i(k) \in \{\text{Explorer, Surveyor, Defender, Idle}\}.$$

The role evolves over time according to a stochastic, context-dependent transition law. The chemical-reaction-network interpretation is used carefully:

- roles behave like internal species,
- role transitions behave like stochastic reactions,
- role populations can be tracked statistically over time.

This is not claimed to be an exact chemical-kinetics derivation. It is a principled modeling analogy for adaptive task allocation and heterogeneous swarm behavior.

5. Mathematical Model

5.1 RGB image to scalar utility field

Let the input satellite image be

$$I(x, y) = (R(x, y), G(x, y), B(x, y)), \quad R, G, B \in [0, 1]$$

Because the image is RGB-only, true NDVI cannot be computed. Instead, the notebook uses the Visible Atmospherically Resistant Index:

$$\text{VARI}(x, y) = \frac{G(x, y) - R(x, y)}{G(x, y) + R(x, y) - B(x, y) + \varepsilon}$$

where

$$\varepsilon > 0$$

prevents division by zero.

The raw VARI field is clipped and normalized into a scalar utility field:

$$\phi(x, y) = \frac{\text{VARI}(x, y) - \text{VARI}_{\min}}{\text{VARI}_{\max} - \text{VARI}_{\min} + \varepsilon}, \quad \phi(x, y) \in [0, 1]$$

The utility field has three roles:

1. It is the reward landscape for PPO.
2. It is the source of local observations.
3. It is the macroscopic coverage objective for the swarm.

5.2 Local observation model

Each agent observes a local square crop centered at its current position:

$$o_i(k) \in \mathbb{R}^{1 \times P \times P}, \quad P = 128$$

The crop is extracted from the utility field with padding near boundaries. This makes the PPO policy local and partially observable.

The local observation window is not only a mathematical object. It must also be visually shown in rollout plots and animations. Every agent should have a visible 128×128 footprint during multi-agent visualization.

5.3 Single-agent PPO reward

Let V be the visited-cell map for an episode. If the agent reaches cell c_k at time step k , the reward is

$$r(k) = \begin{cases} \phi(c_k), & \text{if } c_k \text{ is visited for the first time,} \\ 0, & \text{otherwise.} \end{cases}$$

This reward is intentionally simple. It encourages the agent to discover new cells while preferring vegetation-rich regions.

5.4 PPO microscopic controller

The trained PPO policy maps a local observation to a discrete action:

$$\pi_\theta(o_i(k)) = a_i^{\text{ppo}}(k)$$

where

$$a_i^{\text{ppo}}(k) \in \{\text{up, right, down, left}\}.$$

For the hybrid swarm controller, this discrete action is mapped into a direction vector:

$$d_i^{\text{ppo}}(k) \in \mathbb{R}^2$$

The PPO contribution becomes

$$u_i^{\text{ppo}}(k) = \alpha_{\text{ppo}}(i, k) d_i^{\text{ppo}}(k)$$

This is interpreted as a learned local directional tendency, not as a complete multi-agent coordination law.

5.5 Proximity graph

At each time step, define a proximity graph

$$G(k) = (V, E(k))$$

where an edge exists if two agents are within communication range:

$$(i, j) \in E(k) \iff \|p_i(k) - p_j(k)\|_2 \leq R_{\text{comm}}$$

From this graph, define:

$$A(k) = \text{adjacency matrix,}$$

$$D(k) = \text{degree matrix,}$$

$$L(k) = D(k) - A(k).$$

The communication graph must be visually shown using dotted edges between currently connected agents.

5.6 Consensus correction

A direct position-consensus law,

$$u_i^{\text{cons}}(k) = k_{\text{cons}} \sum_{j \in \mathcal{N}_i(k)} (p_j(k) - p_i(k))$$

can collapse the swarm if applied too strongly. Therefore, the consensus layer is treated as a local coordination smoother.

In implementation, the correction acts on local preferred direction vectors:

$$u_i^{\text{cons}}(k) = k_{\text{cons}} \sum_{j \in \mathcal{N}_i(k)} (d_j(k) - d_i(k))$$

This reduces local directional disagreement while avoiding pure rendezvous behavior.

5.7 Artificial potential-field component

The potential-field contribution is

$$u_i^{\text{pf}} = F_i^{\text{att}} + F_i^{\text{rep}} + F_i^{\text{visit}} + F_i^{\text{bnd}}$$

Utility-gradient attraction

$$F_i^{\text{att}} = k_{\text{att}} \nabla \phi(p_i)$$

This encourages motion toward locally increasing utility.

Inter-agent repulsion For nearby agents,

$$F_i^{\text{rep}} = \sum_{j \neq i} k_{\text{rep}} \psi(\|p_i - p_j\|) \frac{p_i - p_j}{\|p_i - p_j\| + \varepsilon}$$

with a finite-range kernel such as

$$\psi(r) = \max \left(0, \frac{1}{r} - \frac{1}{R_{\text{rep}}} \right)$$

This discourages clustering and close encounters.

Visited-region repulsion Let M_{visit} be the visit-density map. Then

$$F_i^{\text{visit}} = -k_{\text{visit}} \nabla M_{\text{visit}}(p_i)$$

This pushes agents away from over-visited regions.

Boundary shaping A soft inward boundary force is applied near the domain edges:

$$F_i^{\text{bnd}} = \text{inward boundary shaping term}$$

This helps prevent agents from getting trapped against image borders.

5.8 Role-conditioned control

Each agent has a current role:

$$\text{role}_i(k) \in \{\text{Explorer, Surveyor, Defender, Idle}\}.$$

The role changes the weights in the hybrid controller.

Explorer Explorer agents emphasize forward discovery.

Typical behavior:

- stronger PPO drive,
- stronger utility-seeking tendency,
- weaker consensus,
- moderate repulsion.

Surveyor Surveyor agents emphasize systematic inspection and reduced redundancy.

Typical behavior:

- balanced PPO and potential-field terms,
- stronger revisit avoidance,
- moderate consensus,
- stable coverage behavior.

Defender Defender agents emphasize spacing and local stabilization.

Typical behavior:

- lower PPO drive,
- stronger repulsion,
- stronger stabilizing influence,
- improved local separation.

Idle Idle agents reduce unnecessary motion and help avoid over-synchronized behavior.

Typical behavior:

- reduced motion probability,
- weak exploratory drift,
- lower control magnitude,
- congestion relief.

The final hybrid action is

$$u_i(k) = w_{\text{ppo}}(i, k) u_i^{\text{ppo}}(k) + w_{\text{pf}}(i, k) u_i^{\text{pf}}(k) + w_{\text{cons}}(i, k) u_i^{\text{cons}}(k)$$

The control input is saturated:

$$u_i(k) \leftarrow \text{sat}_{u_{\max}}(u_i(k))$$

and the position is updated:

$$p_i(k+1) = p_i(k) + \Delta t u_i(k)$$

6. Role-Color Rule for Visualization

Role switching must be visually legible.

Each agent should not have one permanent trajectory color. Instead, the trajectory color must indicate the agent's current role at that timestep.

A recommended role-color mapping is:

Explorer $\rightarrow$ blue,

Surveyor $\rightarrow$ orange,

Defender $\rightarrow$ green,

Idle $\rightarrow$ red.

This means that a single agent trajectory may contain multiple colored segments. The color of segment

$$p_i(k) \rightarrow p_i(k+1)$$

should correspond to

$$\text{role}_i(k) \text{ or } \text{role}_i(k+1),$$

as long as the convention is stated clearly and used consistently.

This visualization rule is important because role switching is not just a hidden internal variable. It is part of the controller and must be visible in the final outputs.

The notebook should therefore include:

- role-colored trajectory overlays,
- a role legend,
- role-population curves over time,
- final role counts,
- and animation frames where current role is visible through color.

7. Controlled Random Spawning

Random spawning needs to be handled carefully because uncontrolled initialization can make the rollout misleading or unstable.

The pipeline will use controlled random spawning with the following rules:

1. Agents are sampled randomly within the valid image domain.
2. Spawn positions are kept away from hard image boundaries by at least half the observation-window size when possible.
3. Agents must satisfy a minimum initial separation distance:

$$\|p_i(0) - p_j(0)\|_2 \geq d_{\text{spawn},\min}, \quad i \neq j$$

1. The sampler uses rejection sampling with a maximum number of attempts.
2. If strict non-overlapping placement fails, the code should either relax the threshold in a controlled way or raise a clear warning.
3. All initial positions must be logged and printed.
4. Experiments that need reproducibility should use a fixed seed.
5. Visualization-only reruns may optionally use fresh random seeds, but the start positions must still be reported.

The spawn logic should avoid accidental clumping, boundary-biased starts, and repeated identical visual rollouts unless reproducibility is explicitly intended.

The initial swarm state should therefore be recorded as

$$P(0) = \{p_1(0), p_2(0), \dots, p_N(0)\}$$

and stored in the rollout dictionary together with seed information and configuration parameters.

8. How the Notebook Will Proceed

The notebook will be built in a strict sequence.

Step 1: Setup

Import libraries, set paths, configure reproducibility, define global constants, and create output directories.

Step 2: Image ingestion and utility-field construction

Load the RGB satellite image, compute the VARI field, normalize it to $[0, 1]$, optionally smooth it, and visualize:

- original RGB image,
- raw utility field,
- smoothed utility field,
- utility histogram.

Step 3: Single-agent local observation environment

Define a Gymnasium environment where the agent receives a local 128×128 crop and moves using four discrete actions.

Verify:

- observation shape,
- observation dtype,
- reward logic,
- boundary handling,
- local crop alignment.

Step 4: Local observation diagnostic

Explicitly compare the rendered observation window with the extracted local patch to verify that the crop operator is geometrically correct.

Step 5: PPO setup and training

Wrap the environment for Stable-Baselines3, initialize PPO with a CNN policy, train the model, log episode rewards and episode lengths, and save the trained policy.

Step 6: Single-agent evaluation

Evaluate PPO against random behavior over multiple fresh starts. Report:

- total reward,
- utility gain,
- coverage ratio,
- path length,
- net displacement,
- exploration efficiency.

Step 7: Naive multi-agent PPO baseline

Replicate the trained PPO policy across multiple agents without swarm shaping. This establishes the baseline failure mode or limitation of pure policy replication.

Step 8: Potential-field swarm layer

Add utility-gradient attraction, inter-agent repulsion, revisit repulsion, and boundary shaping. Compare against naive PPO.

Step 9: Graph-based coordination layer

Construct the proximity graph, draw communication edges, compute graph diagnostics, and add consensus-like local direction smoothing.

Step 10: Role-switching layer

Add stochastic roles:

Explorer, Surveyor, Defender, Idle.

Use the current role to modulate controller weights and record role history for each agent at every timestep.

Step 11: Full hybrid rollout

Run the complete controller:

PPO + APF + graph coordination + role switching

Store:

- trajectories,
- role history,
- graph history,
- visit map,
- reward history,
- consensus diagnostics,
- role counts,
- spacing diagnostics.

Step 12: Metrics and ablations

Compare controllers using:

$$\text{Coverage Ratio} = \frac{\text{unique visited cells}}{\text{total cells}},$$

$$\text{Utility Gain} = \sum_{\text{first visits}} \phi(c),$$

$$\text{Redundancy Index} = \frac{\text{revisit count}}{\text{total motion steps}}.$$

plus:

- mean reward per agent,
- minimum inter-agent distance,
- close encounter count,
- geometric intersection count,
- mean pairwise distance,
- consensus error,
- graph degree statistics,
- role occupancy fractions.

Step 13: Final visualization

Generate final plots and animation artifacts with:

- visible 128×128 local observation windows,
- visible APF direction cues,
- visible dotted graph edges,
- role-colored trajectory segments,
- role-population plots,
- visit-count heatmaps,
- final metrics summary.

The final animation should make the controller structure visually obvious rather than hidden behind equations.

9. Expected Outputs

By the end of the notebook, the pipeline should produce:

1. Input RGB satellite image visualization.
2. VARI-based utility-field visualization.
3. Utility-field histogram.
4. Local observation sanity checks.
5. PPO training curves.
6. Single-agent PPO evaluation metrics.
7. Random-policy baseline comparison.
8. Naive multi-agent PPO rollout.
9. PPO + potential-field rollout.
10. PPO + potential-field + graph-coordination rollout.
11. Full hybrid rollout with stochastic roles.
12. Role-colored trajectory visualization.
13. Persistent 128×128 local observation windows.
14. Dotted graph edges between communicating agents.
15. APF force or heading cues.
16. Visit-count heatmap.
17. Coverage and redundancy plots.
18. Consensus-error diagnostics.
19. Role-population dynamics over time.
20. Final metrics table.
21. Exported metrics files.
22. Final animation or GIF.

The expected qualitative behavior is:

- PPO alone should show useful local vegetation-aware motion.
- Naive multi-agent PPO may produce redundant or poorly coordinated behavior.
- Potential fields should improve spacing and reduce clustering.

- Graph coordination should improve local coherence and visual swarm structure.
- Role switching should introduce adaptive heterogeneity and make the swarm less homogeneous.
- The final visualization should show a coordinated field system, not a set of independent policy clones.

10. Technical Honesty

This notebook makes several boundaries explicit.

First, VARI is not true NDVI. It is an RGB-derived vegetation proxy and should be interpreted as a practical utility field, not as calibrated multispectral vegetation measurement.

Second, the full hybrid closed-loop system is not claimed to have a complete global convergence proof. Individual components are theoretically motivated, but the complete controller is evaluated empirically.

Third, artificial potential fields can suffer from local minima, oscillations, and narrow-passage issues. In this pipeline, they are used as shaping terms around PPO, not as the sole planner.

Fourth, naive position consensus can collapse a swarm. The graph layer must be used as a local coordination smoother, not as an uncontrolled rendezvous law.

Fifth, CRN-inspired role switching is a modeling framework for stochastic task adaptation. It should not be overclaimed as an exact chemical kinetics model.

Sixth, visual legibility is part of the technical result. If local perception, APF shaping, graph coordination, and role switching are mathematically present but visually invisible, the demonstration is incomplete.

11. Final Thesis

Pure reinforcement learning is strong at learning local navigation behavior from raw local observations, but weak at scalable multi-agent coordination.

Classical multi-robot systems theory is strong at imposing structure, spacing, agreement, coverage geometry, and interpretable collective behavior, but can be brittle when used alone.

The most compelling controller is therefore hybrid:

learned microscopic intelligence + principled macroscopic coordination

This notebook builds that controller end to end.

Section 0: Base Setup, Reproducibility, and Project Configuration

This section initializes the computational scaffold for the final PPO-driven hybrid swarm-control pipeline.

No environment is created in this step.

No PPO training is performed in this step.

No swarm rollout is performed in this step.

The purpose of this section is to define the global experimental structure that every later section will use.

0.1 System Objective

The notebook will implement a hybrid multi-robot coverage controller over a scalar vegetation-utility field.

The final system has the form

$$\text{RGB Satellite Image} \rightarrow \phi(x, y) \rightarrow \pi_{\theta}(o_i) \rightarrow u_i^{\text{hybrid}}$$

Here, $\phi(x, y) \in [0, 1]$

is the normalized utility field derived from the RGB image, and $\pi_{\theta}(o_i)$ is a PPO policy trained on local 128×128 observations.

The full swarm controller will later combine learned local motion with classical multi-robot coordination:

$$u_i(k) = w_{\text{ppo}}(i, k)u_i^{\text{ppo}}(k) + w_{\text{pf}}(i, k)u_i^{\text{pf}}(k) + w_{\text{cons}}(i, k)u_i^{\text{cons}}(k)$$

The current section only prepares the infrastructure needed to make this pipeline reproducible.

0.2 Reproducibility

A fixed random seed is used for:

```
random, numpy, torch, stable-baselines3
```

This matters because the later pipeline will include:

- random single-agent spawn locations,
- controlled random multi-agent spawn locations,
- PPO stochastic policy optimization,
- stochastic role switching,
- randomized rollout diagnostics.

The seed does not eliminate all possible hardware-level nondeterminism, especially on GPU, but it gives a controlled baseline for repeatable experiments.

0.3 Directory Structure

The notebook creates a local project structure:

```
data/
```

for input images and processed utility fields,

```
models/
```

for trained PPO models,

```
results/
```

for plots, metrics, frames, and GIFs,

```
logs/
```

for configuration snapshots and training logs.

This keeps the pipeline reproducible and prevents later sections from silently scattering outputs.

0.4 Global Configuration

A single configuration dataclass stores the main experiment parameters.

Important values include:

$$P = 128$$

for the local observation window size,

$$N = 8$$

for the number of swarm agents,

$$R_{\text{comm}}$$

for graph communication radius,

$$R_{\text{rep}}$$

for short-range repulsion, and

$$d_{\text{spawn},\text{min}}$$

for controlled non-overlapping random spawning.

The spawn-distance parameter is included immediately because uncontrolled random spawning was a known weakness. Later sections will use rejection sampling so that agents do not accidentally begin in an artificially clustered or boundary-biased configuration.

0.5 Role Visualization Convention

The swarm controller will later use stochastic role switching:

$$\text{role}_i(k) \in \{\text{Explorer}, \text{Surveyor}, \text{Defender}, \text{Idle}\}$$

The role is not merely an internal hidden state. It must be visible in the rollout.

Therefore, the role-color convention is fixed here:

Explorer $\rightarrow$ blue,

Surveyor → orange,

Defender → green,

Idle → red.

During final visualization, the color of an agent or trajectory segment must indicate the current role at that timestep.

0.6 What This Cell Should Produce

After running the code cell below, the notebook should print:

- project directory paths,
- device information,
- package versions,
- random seed,
- key PPO parameters,
- key swarm parameters,
- visualization flags,
- role-color mapping,
- and the saved configuration path.

This confirms that the notebook scaffold is ready before constructing the utility field.

```
from __future__ import annotations

import os
import json
import math
import time
import random
from dataclasses import dataclass, asdict
from pathlib import Path

import numpy as np
import cv2
import matplotlib.pyplot as plt

import gymnasium as gym
from gymnasium import spaces

import torch
```

```

import stable_baselines3 as sb3
from stable_baselines3 import PP0
from stable_baselines3.common.utils import set_random_seed

# Global reproducibility

SEED = 42

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)
set_random_seed(SEED)

torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False

# Project paths

PROJECT_ROOT = Path.cwd()

DATA_DIR = PROJECT_ROOT / "data"
RESULTS_DIR = PROJECT_ROOT / "results"
MODELS_DIR = PROJECT_ROOT / "models"
LOGS_DIR = PROJECT_ROOT / "logs"

GIFS_DIR = RESULTS_DIR / "gifs"
PLOTS_DIR = RESULTS_DIR / "plots"
METRICS_DIR = RESULTS_DIR / "metrics"
FRAMES_DIR = RESULTS_DIR / "frames"

for path in [
    DATA_DIR,
    RESULTS_DIR,
    MODELS_DIR,
    LOGS_DIR,
    GIFS_DIR,
    PLOTS_DIR,
    METRICS_DIR,
    FRAMES_DIR,
]:
    path.mkdir(parents=True, exist_ok=True)

# Device

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

```

```

# Role-color convention

ROLE_COLORS = {
    "Explorer": "tab:blue",
    "Surveyor": "tab:orange",
    "Defender": "tab:green",
    "Idle": "tab:red",
}

ROLE_NAMES = tuple(ROLE_COLORS.keys())

# Global configuration

@dataclass
class Config:
    # image / preprocessing
    patch_size: int = 128
    vari_epsilon: float = 1e-6
    gaussian_blur_kernel: int = 5
    use_gaussian_smoothing: bool = True

    # single-agent environment
    max_steps_single: int = 300
    action_step_px: int = 1

    # PPO
    total_timesteps: int = 200_000
    learning_rate: float = 3e-4
    gamma: float = 0.99
    ppo_n_steps: int = 2048
    ppo_batch_size: int = 64
    ppo_n_epochs: int = 10

    # swarm
    num_agents: int = 8
    max_steps_swarm: int = 400
    dt: float = 1.0
    comm_radius: float = 120.0
    repulsion_radius: float = 60.0
    spawn_min_separation: float = 80.0
    spawn_margin: int = 64
    spawn_max_attempts: int = 10_000

    # hybrid-control defaults
    u_max: float = 4.0

```

```

k_att: float = 1.00
k_rep: float = 1.75
k_visit: float = 1.25
k_bnd: float = 1.00
k_cons: float = 0.35

# visualization controls
show_patch_windows: bool = True
show_apf_cues: bool = True
show_graph_edges: bool = True
role_color_tracks_current_role: bool = True
gif_seconds_target: int = 60

# outputs
save_plots: bool = True
save_metrics: bool = True
save_gifs: bool = True

CFG = Config()

# Plotting defaults

plt.rcParams["figure.figsize"] = (6, 5)
plt.rcParams["figure.dpi"] = 120
plt.rcParams["axes.grid"] = True

# Save configuration snapshot

config_payload = asdict(CFG)
config_payload["seed"] = SEED
config_payload["role_colors"] = ROLE_COLORS
config_payload["role_names"] = ROLE_NAMES
config_payload["device"] = str(DEVICE)

config_path = LOGS_DIR / "config_section0.json"

with open(config_path, "w", encoding="utf-8") as f:
    json.dump(config_payload, f, indent=2)

# Sanity printout

print("=" * 72)
print("PP0-Driven Swarm Control | Section 0 Setup Ready")
print("=" * 72)
print(f"Project root           : {PROJECT_ROOT}")

```

```

print(f"Data dir                : {DATA_DIR}")
print(f"Results dir              : {RESULTS_DIR}")
print(f"Models dir                  : {MODELS_DIR}")
print(f"Logs dir                    : {LOGS_DIR}")
print(f"Plots dir                    : {PLOTS_DIR}")
print(f"Metrics dir                  : {METRICS_DIR}")
print(f"GIFs dir                    : {GIFS_DIR}")
print(f"Frames dir                   : {FRAMES_DIR}")
print("-" * 72)
print(f"Device                        : {DEVICE}")
if torch.cuda.is_available():
    print(f"GPU                          : {torch.cuda.get_device_name(0)}")
print(f"PyTorch                      : {torch.__version__}")
print(f"Stable-Baselines3            : {sb3.__version__}")
print(f"OpenCV                       : {cv2.__version__}")
print(f"Gymnasium                    : {gym.__version__}")
print(f"Seed                          : {SEED}")
print("-" * 72)
print(f"Patch size                   : {CFG.patch_size}")
print(f"Single-agent max steps       : {CFG.max_steps_single}")
print(f"PP0 total timesteps          : {CFG.total_timesteps}")
print(f"PP0 learning rate            : {CFG.learning_rate}")
print(f"PP0 gamma                     : {CFG.gamma}")
print("-" * 72)
print(f"Number of agents             : {CFG.num_agents}")
print(f"Swarm rollout steps          : {CFG.max_steps_swarm}")
print(f"Communication radius         : {CFG.comm_radius}")
print(f"Repulsion radius             : {CFG.repulsion_radius}")
print(f"Spawn min separation         : {CFG.spawn_min_separation}")
print(f"Spawn margin                  : {CFG.spawn_margin}")
print(f"Spawn max attempts           : {CFG.spawn_max_attempts}")
print("-" * 72)
print(f"Show patch windows           : {CFG.show_patch_windows}")
print(f"Show APF cues                 : {CFG.show_apf_cues}")
print(f"Show graph edges             : {CFG.show_graph_edges}")
print(f"Role color = current role    : {CFG.role_color_tracks_current_role}")
print(f"GIF target seconds           : {CFG.gif_seconds_target}")
print("-" * 72)
print("Role-color convention:")
for role_name, color_name in ROLE_COLORS.items():
    print(f" {role_name:<8s} -> {color_name}")
print("-" * 72)
print(f"Configuration saved to      : {config_path}")
print("=" * 72)

```

PP0-Driven Swarm Control | Section 0 Setup Ready

```
=====
Project root      : /home/twilightpriest/GitHub/PP0-
driven-Swarm-Control/notebook
Data dir         : /home/twilightpriest/GitHub/PP0-
driven-Swarm-Control/notebook/data
Results dir      : /home/twilightpriest/GitHub/PP0-
driven-Swarm-Control/notebook/results
Models dir       : /home/twilightpriest/GitHub/PP0-
driven-Swarm-Control/notebook/models
Logs dir         : /home/twilightpriest/GitHub/PP0-
driven-Swarm-Control/notebook/logs
Plots dir        : /home/twilightpriest/GitHub/PP0-
driven-Swarm-Control/notebook/results/plots
Metrics dir      : /home/twilightpriest/GitHub/PP0-
driven-Swarm-Control/notebook/results/metrics
GIFs dir         : /home/twilightpriest/GitHub/PP0-
driven-Swarm-Control/notebook/results/gifs
Frames dir       : /home/twilightpriest/GitHub/PP0-
driven-Swarm-Control/notebook/results/frames
-----
```

```
-----
Device           : cuda
GPU              : NVIDIA GeForce RTX 4060 Laptop GPU
PyTorch          : 2.6.0+cu124
Stable-Baselines3 : 2.8.0
OpenCV           : 4.13.0
Gymnasium        : 1.2.3
Seed             : 42
-----
```

```
-----
Patch size       : 128
Single-agent max steps : 300
PP0 total timesteps : 200000
PP0 learning rate  : 0.0003
PP0 gamma        : 0.99
-----
```

```
-----
Number of agents   : 8
Swarm rollout steps : 400
Communication radius : 120.0
Repulsion radius    : 60.0
Spawn min separation : 80.0
Spawn margin        : 64
Spawn max attempts  : 10000
-----
```

```

-----
Show patch windows           : True
Show APF cues                : True
Show graph edges             : True
Role color = current role    : True
GIF target seconds           : 60
-----

Role-color convention:
  Explorer -> tab:blue
  Surveyor -> tab:orange
  Defender -> tab:green
  Idle      -> tab:red
-----

Configuration saved to       : /home/twilightpriest/GitHub/PP0-
driven-Swarm-Control/notebook/logs/config_section0.json
=====

```

Section 1: RGB Satellite Image to Scalar Utility Field

This section constructs the scalar field used by the entire pipeline.
Let the uploaded RGB satellite image be

$$I(x, y) = (R(x, y), G(x, y), B(x, y)), \quad R, G, B \in [0, 1]$$

Because the input is RGB-only, we do **not** compute true NDVI. Instead, we compute the Visible Atmospherically Resistant Index:

$$\text{VARI}(x, y) = \frac{G(x, y) - R(x, y)}{G(x, y) + R(x, y) - B(x, y) + \varepsilon}$$

The raw VARI field is clipped and normalized into a utility field

$$\phi(x, y) = \frac{\text{VARI}(x, y) - \text{VARI}_{\min}}{\text{VARI}_{\max} - \text{VARI}_{\min} + \varepsilon}, \quad \phi(x, y) \in [0, 1]$$

This utility field ϕ will later serve as:

1. the local observation substrate for PPO,
2. the first-visit reward landscape,

3. the scalar coverage objective for the swarm.

This cell loads the satellite image, computes the normalized VARI utility field, optionally smooths it, saves it as `ndvi_field.npy`, and visualizes the RGB image, raw utility map, smoothed utility map, and histogram.

```
sat_path = DATA_DIR / "field_satellite.jpg"

if not sat_path.exists():
    raise FileNotFoundError(
        f"Satellite image not found at:\n{sat_path}\n\n"
        "Place your RGB satellite image in the data directory as:\n"
        "field_satellite.jpg"
    )

# Load RGB image

bgr = cv2.imread(str(sat_path))

if bgr is None:
    raise RuntimeError(f"cv2.imread failed to load image: {sat_path}")

rgb = cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)
rgb_float = rgb.astype(np.float32) / 255.0

H_img, W_img, C_img = rgb.shape

print("=" * 72)
print("Section 1 | RGB Image Loaded")
print("=" * 72)
print(f"Image path           : {sat_path}")
print(f"Image shape           : {rgb.shape}")
print(f"Image dtype           : {rgb.dtype}")
print(f"Image height, width   : {H_img}, {W_img}")

# Split channels

R = rgb_float[:, :, 0]
G = rgb_float[:, :, 1]
B = rgb_float[:, :, 2]

# Compute VARI

eps = CFG.vari_epsilon

vari_raw = (G - R) / (G + R - B + eps)
```

```

vari_clipped = np.clip(vari_raw, -1.0, 1.0)

vmin = float(vari_clipped.min())
vmax = float(vari_clipped.max())

phi_raw = (vari_clipped - vmin) / (vmax - vmin + eps)
phi_raw = np.clip(phi_raw, 0.0, 1.0).astype(np.float32)

# Optional smoothing

if CFG.use_gaussian_smoothing:
    k = int(CFG.gaussian_blur_kernel)

    if k < 3:
        k = 3
    if k % 2 == 0:
        k += 1

    phi_smooth = cv2.GaussianBlur(phi_raw, (k, k), sigmaX=0)
else:
    k = None
    phi_smooth = phi_raw.copy()

ndvi_field = np.clip(phi_smooth, 0.0, 1.0).astype(np.float32)

# Save processed field

ndvi_save_path = DATA_DIR / "ndvi_field.npy"
np.save(ndvi_save_path, ndvi_field)

utility_png_path = PLOTS_DIR / "section1_vari_utility_field.png"
hist_png_path = PLOTS_DIR / "section1_utility_histogram.png"

# Print diagnostics

print("-" * 72)
print("VARI / Utility Diagnostics")
print("-" * 72)
print(f"VARI clipped range           : [{vmin:.6f}, {vmax:.6f}]")
print(f"Raw utility min / max         : {phi_raw.min():.6f} / {phi_raw.max():.6f}")
print(f"Final utility min / max       : {ndvi_field.min():.6f} / {ndvi_field.max():.6f}")
print(f"Final utility mean / std      : {ndvi_field.mean():.6f} / {ndvi_field.std():.6f}")
print(f"Gaussian smoothing enabled    : {CFG.use_gaussian_smoothing}")
print(f"Gaussian kernel used          : {k}")
print(f"Saved utility field           : {ndvi_save_path}")

```

```
# Visual diagnostics
```

```
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
```

```
axes[0].imshow(rgb)
axes[0].set_title("Input RGB Satellite Image")
axes[0].axis("off")
```

```
im1 = axes[1].imshow(phi_raw, cmap="viridis", vmin=0.0, vmax=1.0)
axes[1].set_title("Normalized VARI Utility Field")
axes[1].axis("off")
plt.colorbar(im1, ax=axes[1], fraction=0.046, pad=0.04)
```

```
im2 = axes[2].imshow(ndvi_field, cmap="viridis", vmin=0.0, vmax=1.0)
axes[2].set_title("Smoothed Utility Field")
axes[2].axis("off")
plt.colorbar(im2, ax=axes[2], fraction=0.046, pad=0.04)
```

```
plt.tight_layout()
```

```
if CFG.save_plots:
    fig.savefig(utility_png_path, dpi=200, bbox_inches="tight")
    print(f"Saved utility visualization : {utility_png_path}")
```

```
plt.show()
```

```
fig_hist, ax_hist = plt.subplots(figsize=(6, 4))
ax_hist.hist(ndvi_field.ravel(), bins=50)
ax_hist.set_title("Utility Field Histogram")
ax_hist.set_xlabel("Utility value")
ax_hist.set_ylabel("Pixel count")
plt.tight_layout()
```

```
if CFG.save_plots:
    fig_hist.savefig(hist_png_path, dpi=200, bbox_inches="tight")
    print(f"Saved utility histogram : {hist_png_path}")
```

```
plt.show()
```

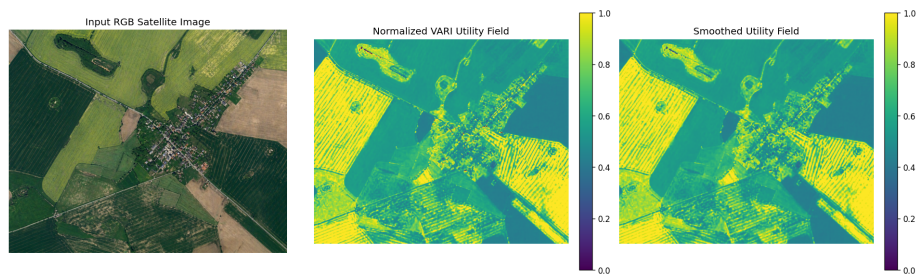
```
print("=" * 72)
print("Section 1 complete: utility field is ready.")
print("=" * 72)
```

```
=====
Section 1 | RGB Image Loaded
=====
```

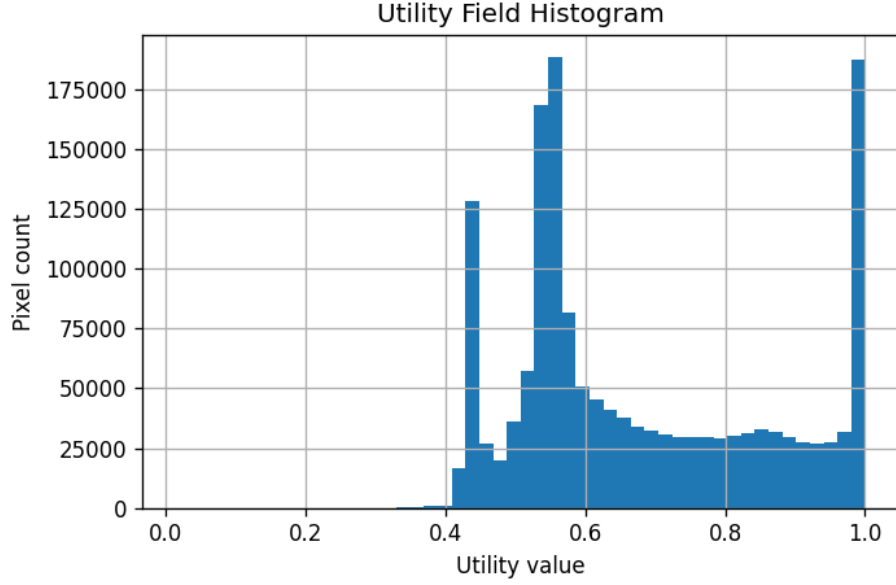
```
Image path          : /home/twilightpriest/GitHub/PP0-  
driven-Swarm-Control/notebook/data/field_satellite.jpg  
Image shape         : (1122, 1402, 3)  
Image dtype         : uint8  
Image height, width : 1122, 1402
```

VARI / Utility Diagnostics

```
VARI clipped range      : [-1.000000, 1.000000]  
Raw utility min / max   : 0.000000 / 1.000000  
Final utility min / max : 0.014873 / 1.000000  
Final utility mean / std : 0.679508 / 0.185383  
Gaussian smoothing enabled : True  
Gaussian kernel used     : 5  
Saved utility field      : /home/twilightpriest/GitHub/PP0-  
driven-Swarm-Control/notebook/data/ndvi_field.npy  
Saved utility visualization : /home/twilightpriest/GitHub/PP0-  
driven-Swarm-Control/notebook/results/plots/section1_vari_utility_field.png
```



```
Saved utility histogram : /home/twilightpriest/GitHub/PP0-  
driven-Swarm-Control/notebook/results/plots/section1_utility_histogram.png
```



=====
 Section 1 complete: utility field is ready.
 =====

Section 2: Single-Agent Local Observation Environment

This section defines the single-agent Gymnasium environment used to train the microscopic PPO controller.

The environment is built on the scalar utility field

$$\phi \in [0, 1]^{H \times W}$$

At timestep k , the agent has pixel position

$$p_k = (y_k, x_k),$$

but the policy does not observe the full map. Instead, it receives a local crop

$$o_k \in \mathbb{R}^{1 \times P \times P}, \quad P = 128.$$

The action space is discrete:

$$\mathcal{A} = \{\text{up, right, down, left}\}.$$

The reward follows the first-visit utility rule:

$$r_k = \begin{cases} \phi(c_k), & \text{if } c_k \text{ is visited for the first time,} \\ 0, & \text{otherwise.} \end{cases}$$

The spawn position is sampled randomly on reset. A fixed seed may make the first reset reproducible, but the environment logic itself uses random spawning, not a hardcoded start location.

This cell defines the environment, runs a short random rollout, renders the agent with its 128×128 observation window, and displays the current local observation patch.

```
class NDVIDroneEnv(gym.Env):
    """
    Single-agent local-observation environment over a VARI-derived utility field

    Observation:
        Local patch of shape (1, patch_size, patch_size), dtype uint8

    Action space:
        0 = up, 1 = right, 2 = down, 3 = left

    Reward:
        Utility value on first visit to a cell, else 0

    Episode ending:
        Fixed time horizon via truncation
    """

    metadata = {"render_modes": ["rgb_array"], "render_fps": 5}

    def __init__(
        self,
        ndvi_field: np.ndarray,
        patch_size: int = 128,
        max_steps: int = 300,
        action_step_px: int = 1,
        spawn_margin: int | None = None,
    ):
        super().__init__()

        assert ndvi_field.ndim == 2, "ndvi_field must be a 2D array."
```

```

self.ndvi_field = ndvi_field.astype(np.float32)
self.H, self.W = self.ndvi_field.shape

self.patch_size = int(patch_size)
self.max_steps = int(max_steps)
self.action_step_px = int(action_step_px)
self.pad = self.patch_size // 2

if spawn_margin is None:
    spawn_margin = self.pad

self.spawn_margin = int(spawn_margin)

# Keep spawn margin valid even for smaller images.
self.y_low = min(max(self.spawn_margin, 0), self.H - 1)
self.y_high = max(min(self.H - self.spawn_margin, self.H), self.y_low + 1)
self.x_low = min(max(self.spawn_margin, 0), self.W - 1)
self.x_high = max(min(self.W - self.spawn_margin, self.W), self.x_low + 1)

self.ndvi_padded = np.pad(
    self.ndvi_field,
    pad_width=self.pad,
    mode="constant",
    constant_values=0.0,
)

self.action_space = spaces.Discrete(4)

self.observation_space = spaces.Box(
    low=0,
    high=255,
    shape=(1, self.patch_size, self.patch_size),
    dtype=np.uint8,
)

self.agent_x: int | None = None
self.agent_y: int | None = None
self.visited: np.ndarray | None = None
self.step_count = 0
self.last_spawn_seed = None

def _random_spawn(self) -> None:
    """
    Randomly sample the starting position.

```

```

This is intentionally not hardcoded. With a fixed reset seed, the sample
location is reproducible; without a fixed seed, it changes across resets
"""
self.agent_y = int(self.np_random.integers(self.y_low, self.y_high))
self.agent_x = int(self.np_random.integers(self.x_low, self.x_high))

def _get_obs(self) -> np.ndarray:
    y, x = self.agent_y, self.agent_x

    yp = y + self.pad
    xp = x + self.pad
    p = self.patch_size

    patch = self.ndvi_padded[
        yp - p // 2 : yp + p // 2 + p,
        xp - p // 2 : xp + p // 2 + p,
    ]

    patch_uint8 = np.clip(patch * 255.0, 0, 255).astype(np.uint8)
    obs = patch_uint8[None, ...]

    return obs

def reset(self, *, seed=None, options=None):
    super().reset(seed=seed)

    self.last_spawn_seed = seed
    self.step_count = 0
    self.visited = np.zeros((self.H, self.W), dtype=bool)

    self._random_spawn()

    self.visited[self.agent_y, self.agent_x] = True

    obs = self._get_obs()

    info = {
        "agent_pos": (self.agent_y, self.agent_x),
        "spawn_seed": seed,
        "spawn_is_random": True,
        "spawn_margin": self.spawn_margin,
        "step_count": self.step_count,
        "utility_value": float(self.ndvi_field[self.agent_y, self.agent_x]),
    }

    return obs, info

```

```

def step(self, action):
    action = int(action)

    if action == 0:        # up
        self.agent_y -= self.action_step_px
    elif action == 1:      # right
        self.agent_x += self.action_step_px
    elif action == 2:      # down
        self.agent_y += self.action_step_px
    elif action == 3:      # left
        self.agent_x -= self.action_step_px
    else:
        raise ValueError(f"Invalid action: {action}")

    self.agent_y = int(np.clip(self.agent_y, 0, self.H - 1))
    self.agent_x = int(np.clip(self.agent_x, 0, self.W - 1))

    y, x = self.agent_y, self.agent_x

    first_visit = not self.visited[y, x]
    reward = float(self.ndvi_field[y, x]) if first_visit else 0.0
    self.visited[y, x] = True

    self.step_count += 1

    terminated = False
    truncated = self.step_count >= self.max_steps

    obs = self._get_obs()

    info = {
        "agent_pos": (y, x),
        "step_count": self.step_count,
        "first_visit": first_visit,
        "utility_value": float(self.ndvi_field[y, x]),
        "unique_visited": int(self.visited.sum()),
    }

    return obs, reward, terminated, truncated, info

def render(self):
    import matplotlib.patches as patches

    fig, ax = plt.subplots(figsize=(6, 6), dpi=120)

```

```

ax.imshow(self.ndvi_field, cmap="viridis", vmin=0.0, vmax=1.0)
ax.axis("off")

ax.scatter(
    [self.agent_x],
    [self.agent_y],
    s=45,
    c="white",
    edgecolors="black",
    linewidths=1.0,
    zorder=4,
)

if CFG.show_patch_windows:
    half = self.patch_size // 2

    rect = patches.Rectangle(
        (self.agent_x - half, self.agent_y - half),
        self.patch_size,
        self.patch_size,
        linewidth=1.8,
        edgecolor="red",
        facecolor="none",
        zorder=3,
    )
    ax.add_patch(rect)

fig.canvas.draw()
rgba = np.asarray(fig.canvas.buffer_rgba())
rgb_frame = rgba[:, :, :3].copy()

plt.close(fig)

return rgb_frame

```

Build environment

```

env = NDVIDroneEnv(
    ndvi_field=ndvi_field,
    patch_size=CFG.patch_size,
    max_steps=CFG.max_steps_single,
    action_step_px=CFG.action_step_px,
    spawn_margin=CFG.spawn_margin,
)

```

```

# Seeded reset: still random sampled, but reproducible.
obs, info = env.reset(seed=SEED)

print("=" * 72)
print("Section 2 | Single-Agent Environment Ready")
print("=" * 72)
print(f"Observation shape           : {obs.shape}")
print(f"Observation dtype           : {obs.dtype}")
print(f"Observation min / max        : {obs.min()} / {obs.max()}")
print(f"Action space                  : {env.action_space}")
print(f"Utility field shape          : {env.ndvi_field.shape}")
print(f"Patch size                    : {env.patch_size}")
print(f"Spawn is random               : {info['spawn_is_random']}")
print(f"Spawn seed                    : {info['spawn_seed']}")
print(f"Spawn margin                  : {info['spawn_margin']}")
print(f"Initial agent position        : {info['agent_pos']}")
print(f"Initial utility value         : {info['utility_value']:.6f}")
print("-" * 72)

# Confirm random spawning across unseeded resets

random_spawn_positions = []

for _ in range(5):
    _, reset_info = env.reset()
    random_spawn_positions.append(reset_info["agent_pos"])

# Return to reproducible state for this section's demo rollout.
obs, info = env.reset(seed=SEED)

print("Five unseeded random spawn samples:")
for idx, pos in enumerate(random_spawn_positions, start=1):
    print(f"    sample {idx}: {pos}")

print(f"Demo rollout reset position : {info['agent_pos']}")
print("-" * 72)

# Short random rollout

print("Random rollout diagnostic:")

for t in range(10):
    action = env.action_space.sample()
    obs, reward, terminated, truncated, info = env.step(action)

    print(

```

```

        f"Step {t + 1:02d} | "
        f"action={action} | "
        f"reward={reward:.6f} | "
        f"first_visit={info['first_visit']} | "
        f"utility={info['utility_value']:.6f} | "
        f"pos={info['agent_pos']} | "
        f"unique={info['unique_visited']}"
    )

    if terminated or truncated:
        print("Episode ended during diagnostic rollout.")
        break

# Render global frame with local window

frame = env.render()

print("-" * 72)
print(f"Rendered frame shape      : {frame.shape}")
print(f"Rendered frame dtype      : {frame.dtype}")

plt.figure(figsize=(6, 6))
plt.imshow(frame)
plt.title("Single-Agent Render with 128x128 Local Observation Window")
plt.axis("off")
plt.tight_layout()
plt.show()

# Show current local observation patch

plt.figure(figsize=(5, 5))
plt.imshow(obs[0], cmap="viridis", vmin=0, vmax=255)
plt.title("Current 128x128 Local Observation Patch")
plt.axis("off")
plt.tight_layout()
plt.show()

print("=" * 72)
print("Section 2 complete: single-agent local environment is ready.")
print("=" * 72)

=====
Section 2 | Single-Agent Environment Ready
=====
Observation shape      : (1, 128, 128)
Observation dtype      : uint8

```

```

Observation min / max      : 122 / 152
Action space               : Discrete(4)
Utility field shape        : (1122, 1402)
Patch size                 : 128
Spawn is random            : True
Spawn seed                 : 42
Spawn margin               : 64
Initial agent position     : (152, 1050)
Initial utility value      : 0.513724

```

Five unseeded random spawn samples:

```

sample 1: (714, 623)
sample 2: (494, 1157)
sample 3: (149, 952)
sample 4: (264, 183)
sample 5: (587, 1306)

```

Demo rollout reset position : (152, 1050)

Random rollout diagnostic:

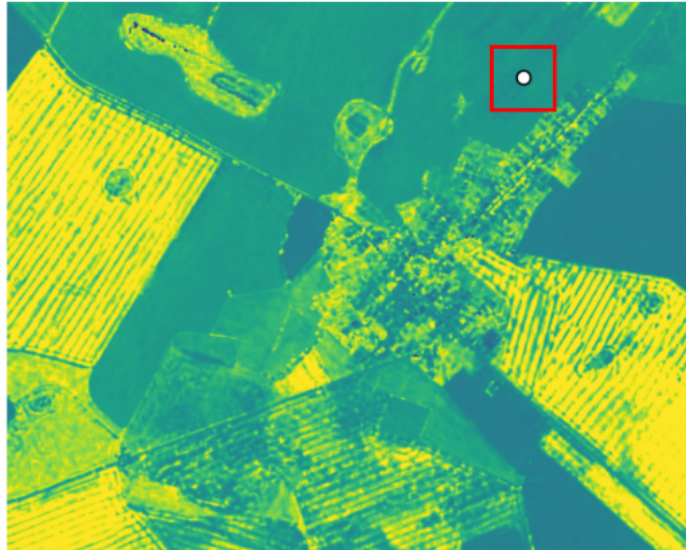
Step 01	action=1	reward=0.513583	first_visit=True	utility=0.513583	pos=(152, 1050)
Step 02	action=2	reward=0.515014	first_visit=True	utility=0.515014	pos=(152, 1050)
Step 03	action=1	reward=0.515552	first_visit=True	utility=0.515552	pos=(152, 1050)
Step 04	action=0	reward=0.513543	first_visit=True	utility=0.513543	pos=(152, 1050)
Step 05	action=2	reward=0.000000	first_visit=False	utility=0.515552	pos=(152, 1050)
Step 06	action=1	reward=0.516405	first_visit=True	utility=0.516405	pos=(152, 1050)
Step 07	action=2	reward=0.519337	first_visit=True	utility=0.519337	pos=(152, 1050)
Step 08	action=3	reward=0.517731	first_visit=True	utility=0.517731	pos=(152, 1050)
Step 09	action=3	reward=0.516265	first_visit=True	utility=0.516265	pos=(152, 1050)
Step 10	action=0	reward=0.000000	first_visit=False	utility=0.515014	pos=(152, 1050)

```

Rendered frame shape      : (720, 720, 3)
Rendered frame dtype      : uint8

```

Single-Agent Render with 128x128 Local Observation Window



Current 128x128 Local Observation Patch



=====
Section 2 complete: single-agent local environment is ready.
=====

Section 3: Local Observation Diagnostic

This section verifies that the local crop returned by the environment is geometrically aligned with the agent position.

For an agent at

$$p_k = (y_k, x_k)$$

the environment should return a crop

$$o_k \in \mathbb{R}^{1 \times 128 \times 128}$$

centered at (y_k, x_k) , with padding only when the crop crosses the image boundary.

We compare two quantities:

1. the observation produced by `env._get_obs()`,
2. an independently reconstructed 128×128 crop from the utility field.

After converting both to uint8, the difference should satisfy

$$\max |o_{\text{env}} - o_{\text{reconstructed}}| = 0.$$

This is a diagnostic-only step. It does not change the environment or the reward.

```
def extract_float_patch_from_field(
    field: np.ndarray,
    y: int,
    x: int,
    patch_size: int,
) -> np.ndarray:
    """
    Independently reconstruct the local float patch centered at (y, x).

    This mirrors the environment crop logic but does not call env._get_obs().
    """
    pad = patch_size // 2

    padded = np.pad(
        field,
        pad_width=pad,
        mode="constant",
        constant_values=0.0,
    )

    yp = y + pad
    xp = x + pad
    p = patch_size

    patch = padded[
        yp - p // 2 : yp + p // 2 + p,
        xp - p // 2 : xp + p // 2 + p,
    ]
```

```

        return patch.astype(np.float32)

# Current agent state
y_curr, x_curr = env.agent_y, env.agent_x

# Patch reconstructed directly from the utility field
patch_float = extract_float_patch_from_field(
    field=ndvi_field,
    y=y_curr,
    x=x_curr,
    patch_size=CFG.patch_size,
)

# Patch returned by the environment observation path
obs_patch_uint8 = env._get_obs()[0]

# Reconstruct the expected uint8 version from the float patch
patch_from_float_uint8 = np.clip(
    patch_float * 255.0,
    0,
    255,
).astype(np.uint8)

# Difference image
abs_diff = np.abs(
    obs_patch_uint8.astype(np.int16)
    -
    patch_from_float_uint8.astype(np.int16)
)

# Numeric diagnostics

print("=" * 72)
print("Section 3 | Local Observation Diagnostic")
print("=" * 72)
print(f"Current agent position          : {(y_curr, x_curr)}")
print(f"Patch size                      : {CFG.patch_size}")
print(f"Float patch shape                : {patch_float.shape}")
print(f"Env obs patch shape              : {obs_patch_uint8.shape}")
print(f"Max absolute uint8 diff          : {abs_diff.max()}")
print(f"Mean absolute uint8 diff         : {abs_diff.mean():.8f}")
print(f"Float patch min / max            : {patch_float.min():.6f} / {patch_float.max()}")
print(f"Env obs uint8 min / max          : {obs_patch_uint8.min()} / {obs_patch_uint8.max()}")

```

```

if abs_diff.max() == 0:
    print("Diagnostic result           : PASS - crop logic is exactly aligned.")
else:
    print("Diagnostic result           : WARNING - crop mismatch detected.")

# Visual comparison

frame = env.render()

fig, axes = plt.subplots(1, 3, figsize=(15, 5))

axes[0].imshow(frame)
axes[0].set_title("Global Render with Local Window")
axes[0].axis("off")

axes[1].imshow(patch_float, cmap="viridis", vmin=0.0, vmax=1.0)
axes[1].set_title("Reconstructed Float Patch")
axes[1].axis("off")

axes[2].imshow(obs_patch_uint8, cmap="viridis", vmin=0, vmax=255)
axes[2].set_title("Environment Observation Patch")
axes[2].axis("off")

plt.tight_layout()
plt.show()

# Difference visualization

plt.figure(figsize=(5, 5))
plt.imshow(abs_diff, cmap="hot")
plt.title("| Env Observation - Reconstructed Patch |")
plt.axis("off")
plt.tight_layout()
plt.show()

print("=" * 72)
print("Section 3 complete: local observation geometry verified.")
print("=" * 72)

```

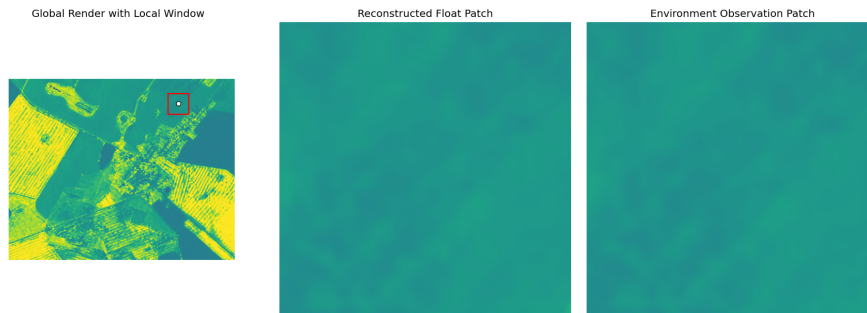
Section 3 | Local Observation Diagnostic

```

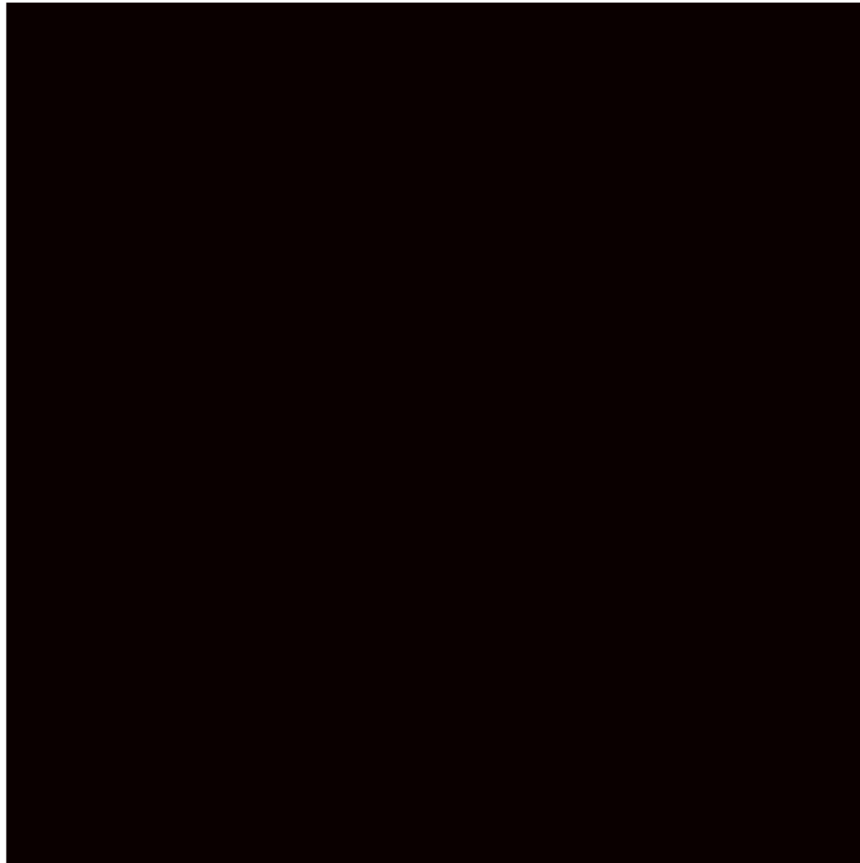
Current agent position      : (153, 1051)
Patch size                  : 128
Float patch shape           : (128, 128)
Env obs patch shape         : (128, 128)

```

Max absolute uint8 diff : 0
Mean absolute uint8 diff : 0.00000000
Float patch min / max : 0.480874 / 0.623192
Env obs uint8 min / max : 122 / 158
Diagnostic result : PASS - crop logic is exactly aligned.



| Env Observation - Reconstructed Patch |



=====
Section 3 complete: local observation geometry verified.
=====

Section 4: Vectorized Environment Interface for PPO

Stable-Baselines3 expects a vectorized environment interface.

The base environment returns one observation with shape

$$(1, P, P), \quad P = 128$$

After wrapping with DummyVecEnv, the observation becomes

$$(N_{\text{env}}, 1, P, P)$$

Here,

$$N_{\text{env}} = 1$$

so the expected vectorized observation shape is

$$(1, 1, 128, 128)$$

This section only verifies the PPO interface. It does not train the policy yet.

The goals are:

1. wrap the local-observation environment,
2. confirm the batched observation shape and dtype,
3. initialize a CNN-based PPO policy without tensor-shape errors.

```
from stable_baselines3.common.vec_env import DummyVecEnv
```

```
def make_env(seed: int | None = None):  
    """  
    Factory function for Stable-Baselines3 vectorized environments.  
  
    The environment still uses random spawning internally.  
    If seed is provided, the first reset is reproducible.  
    """
```

```

def _init():
    env_local = NDVIDroneEnv(
        ndvi_field=ndvi_field,
        patch_size=CFG.patch_size,
        max_steps=CFG.max_steps_single,
        action_step_px=CFG.action_step_px,
        spawn_margin=CFG.spawn_margin,
    )
    env_local.reset(seed=seed)
    return env_local

return _init

# Single-environment vectorized wrapper

vec_env = DummyVecEnv([make_env(SEED)])

vec_obs = vec_env.reset()

print("=" * 72)
print("Section 4 | Vectorized PPO Interface")
print("=" * 72)
print(f"Vectorized observation shape : {vec_obs.shape}")
print(f"Vectorized observation dtype : {vec_obs.dtype}")
print(f"Expected shape                : {(1, 1, CFG.patch_size, CFG.patch_size)}")
print(f"Observation min / max          : {vec_obs.min()} / {vec_obs.max()}")

shape_ok = vec_obs.shape == (1, 1, CFG.patch_size, CFG.patch_size)
dtype_ok = vec_obs.dtype == np.uint8

print(f"Shape check                      : {'PASS' if shape_ok else 'FAIL'}")
print(f"Dtype check                      : {'PASS' if dtype_ok else 'FAIL'}")

if not shape_ok:
    raise ValueError(
        f"Unexpected VecEnv observation shape: {vec_obs.shape}. "
        f"Expected {(1, 1, CFG.patch_size, CFG.patch_size)}."
    )

if not dtype_ok:
    raise TypeError(
        f"Unexpected VecEnv observation dtype: {vec_obs.dtype}. "
        f"Expected np.uint8."
    )

# PPO model initialization only

```

```

ppo_model = PPO(
    policy="CnnPolicy",
    env=vec_env,
    learning_rate=CFG.learning_rate,
    gamma=CFG.gamma,
    n_steps=CFG.ppo_n_steps,
    batch_size=CFG.ppo_batch_size,
    n_epochs=CFG.ppo_n_epochs,
    verbose=1,
    device=DEVICE,
)

print("-" * 72)
print("PP0 model initialized successfully.")
print(f"Policy class           : {ppo_model.policy.__class__.__name__}")
print(f"Policy device          : {ppo_model.device}")
print(f"Learning rate           : {CFG.learning_rate}")
print(f"Gamma                   : {CFG.gamma}")
print(f"n_steps                 : {CFG.ppo_n_steps}")
print(f"Batch size              : {CFG.ppo_batch_size}")
print(f"n_epochs                : {CFG.ppo_n_epochs}")
print("=" * 72)
print("Section 4 complete: PP0 interface is valid.")
print("=" * 72)

=====
Section 4 | Vectorized PP0 Interface
=====
Vectorized observation shape : (1, 1, 128, 128)
Vectorized observation dtype : uint8
Expected shape               : (1, 1, 128, 128)
Observation min / max       : 114 / 254
Shape check                  : PASS
Dtype check                  : PASS
Using cuda device

-----
PP0 model initialized successfully.
Policy class                 : ActorCriticCnnPolicy
Policy device                : cuda
Learning rate                 : 0.0003
Gamma                        : 0.99
n_steps                      : 2048
Batch size                   : 64
n_epochs                     : 10

```

=====
Section 4 complete: PPO interface is valid.
=====

Section 5: PPO Training with Sparse Logged Learning Curves

This section trains the microscopic PPO controller.

The policy is

$$\pi_{\theta}(a_k \mid o_k)$$

where the observation is the local crop

$$o_k \in \mathbb{R}^{1 \times 128 \times 128}$$

and the action is one of

$$\{\text{up, right, down, left}\}$$

The reward remains the first-visit utility reward:

$$r_k = \begin{cases} \phi(c_k), & \text{if } c_k \text{ is visited for the first time,} \\ 0, & \text{otherwise.} \end{cases}$$

This trains PPO as a local navigation primitive only. It is not yet a swarm controller.

To keep the notebook output readable, PPO console verbosity is reduced. Instead of printing logs every rollout, a custom callback prints a compact progress summary every 40,000 timesteps.

```
from stable_baselines3.common.callbacks import BaseCallback
from stable_baselines3.common.monitor import Monitor
```

```
class SparseEpisodeLoggingCallback(BaseCallback):
```

```
    """
```

```
    Collect episode statistics and print compact training progress sparsely.
```

```
    SB3's default verbose logs can be too noisy because they print every rollout
    This callback prints only every `print_freq` environment steps.
```

```

"""

def __init__(self, print_freq: int = 40_000):
    super().__init__()
    self.print_freq = int(print_freq)
    self.episode_rewards = []
    self.episode_lengths = []
    self.last_print_step = 0

def _on_step(self) -> bool:
    infos = self.locals.get("infos", [])

    for info in infos:
        if "episode" in info:
            self.episode_rewards.append(float(info["episode"]["r"]))
            self.episode_lengths.append(int(info["episode"]["l"]))

    current_step = int(self.num_timesteps)

    if current_step - self.last_print_step >= self.print_freq:
        self.last_print_step = current_step

        if len(self.episode_rewards) > 0:
            recent_rewards = self.episode_rewards[-20:]
            recent_lengths = self.episode_lengths[-20:]

            mean_recent_reward = float(np.mean(recent_rewards))
            mean_recent_length = float(np.mean(recent_lengths))

            print(
                f"[PP0 progress] "
                f"timesteps={current_step:>7d} | "
                f"episodes={len(self.episode_rewards):>4d} | "
                f"mean_reward_last20={mean_recent_reward:>8.3f} | "
                f"mean_len_last20={mean_recent_length:>6.1f}"
            )
        else:
            print(
                f"[PP0 progress] "
                f"timesteps={current_step:>7d} | "
                f"episodes=0 | waiting for completed episodes"
            )

    return True

```

```

def make_monitored_env(seed: int | None = None):
    """
    Create a monitored environment so SB3 exposes episode reward and length.

    The environment uses random spawning at reset. A seed gives reproducible
    stochastic initialization for controlled training runs.
    """
    def _init():
        env_local = NDVIDroneEnv(
            ndvi_field=ndvi_field,
            patch_size=CFG.patch_size,
            max_steps=CFG.max_steps_single,
            action_step_px=CFG.action_step_px,
            spawn_margin=CFG.spawn_margin,
        )
        env_local = Monitor(env_local)
        env_local.reset(seed=seed)
        return env_local

    return _init

# Rebuild VecEnv with Monitor
vec_env = DummyVecEnv([make_monitored_env(SEED)])

# PPO model for training
ppo_model = PPO(
    policy="CnnPolicy",
    env=vec_env,
    learning_rate=CFG.learning_rate,
    gamma=CFG.gamma,
    n_steps=CFG.ppo_n_steps,
    batch_size=CFG.ppo_batch_size,
    n_epochs=CFG.ppo_n_epochs,
    verbose=0, # keep notebook logs clean
    device=DEVICE,
    tensorboard_log=str(LOGS_DIR / "tb"),
)

episode_logger = SparseEpisodeLoggingCallback(print_freq=40_000)

print("=" * 72)
print("Section 5 | PPO Training")
print("=" * 72)
print(f"Total timesteps           : {CFG.total_timesteps}")

```

```

print(f"Progress print frequency      : {episode_logger.print_freq}")
print(f"Policy                        : CnnPolicy")
print(f"Device                        : {ppo_model.device}")
print(f"Environment max steps           : {CFG.max_steps_single}")
print(f"Reward                          : first-visit utility")
print("-" * 72)

train_start = time.time()

ppo_model.learn(
    total_timesteps=CFG.total_timesteps,
    callback=episode_logger,
    progress_bar=False,
)

train_end = time.time()
training_time = train_end - train_start

# Save trained model

model_path = MODELS_DIR / "ppo_ndvi_drone_final"
ppo_model.save(str(model_path))

# Training summary

num_logged = len(episode_logger.episode_rewards)

print("-" * 72)
print("Training finished.")
print(f"Training time                : {training_time:.2f} s")
print(f"Logged episodes              : {num_logged}")
print(f"Saved PPO model              : {model_path}.zip")

if num_logged > 0:
    rewards = np.array(episode_logger.episode_rewards, dtype=np.float32)
    lengths = np.array(episode_logger.episode_lengths, dtype=np.float32)

    print(f"First episode reward        : {rewards[0]:.4f}")
    print(f>Last episode reward        : {rewards[-1]:.4f}")
    print(f"Mean reward                 : {rewards.mean():.4f}")
    print(f"Mean last 20 rewards        : {rewards[-20:].mean():.4f}")
    print(f"Mean episode length         : {lengths.mean():.2f}")

# Save raw learning curves
training_metrics = {
    "episode_rewards": episode_logger.episode_rewards,

```

```

        "episode_lengths": episode_logger.episode_lengths,
        "training_time_sec": training_time,
        "total_timesteps": CFG.total_timesteps,
        "print_frequency": episode_logger.print_freq,
    }

    training_metrics_path = METRICS_DIR / "section5_ppo_training_metrics.json"
    with open(training_metrics_path, "w", encoding="utf-8") as f:
        json.dump(training_metrics, f, indent=2)

    print(f"Saved training metrics      : {training_metrics_path}")

    # Reward curve
    reward_plot_path = PLOTS_DIR / "section5_ppo_episode_reward.png"

    fig_reward, ax_reward = plt.subplots(figsize=(7, 4))
    ax_reward.plot(rewards)
    ax_reward.set_title("PPO Training: Episode Reward")
    ax_reward.set_xlabel("Episode index")
    ax_reward.set_ylabel("Episode reward")
    plt.tight_layout()

    if CFG.save_plots:
        fig_reward.savefig(reward_plot_path, dpi=200, bbox_inches="tight")
        print(f"Saved reward plot      : {reward_plot_path}")

    plt.show()

    # Episode length curve
    length_plot_path = PLOTS_DIR / "section5_ppo_episode_length.png"

    fig_length, ax_length = plt.subplots(figsize=(7, 4))
    ax_length.plot(lengths)
    ax_length.set_title("PPO Training: Episode Length")
    ax_length.set_xlabel("Episode index")
    ax_length.set_ylabel("Episode length")
    plt.tight_layout()

    if CFG.save_plots:
        fig_length.savefig(length_plot_path, dpi=200, bbox_inches="tight")
        print(f"Saved length plot      : {length_plot_path}")

    plt.show()

else:
    print("No episode statistics were logged.")

```

```

print("=" * 72)
print("Section 5 complete: PP0 local navigation policy is trained.")
print("=" * 72)

```

Section 5 | PP0 Training

```

Total timesteps           : 200000
Progress print frequency  : 40000
Policy                    : CnnPolicy
Device                    : cuda
Environment max steps     : 300
Reward                    : first-visit utility

```

```

-----
[PP0 progress] timesteps= 40000 | episodes= 133 | mean_reward_last20= 191.931 | mea
[PP0 progress] timesteps= 80000 | episodes= 266 | mean_reward_last20= 206.450 | mea
[PP0 progress] timesteps= 120000 | episodes= 400 | mean_reward_last20= 224.401 | mea
[PP0 progress] timesteps= 160000 | episodes= 533 | mean_reward_last20= 202.136 | mea
[PP0 progress] timesteps= 200000 | episodes= 666 | mean_reward_last20= 215.224 | mea
-----

```

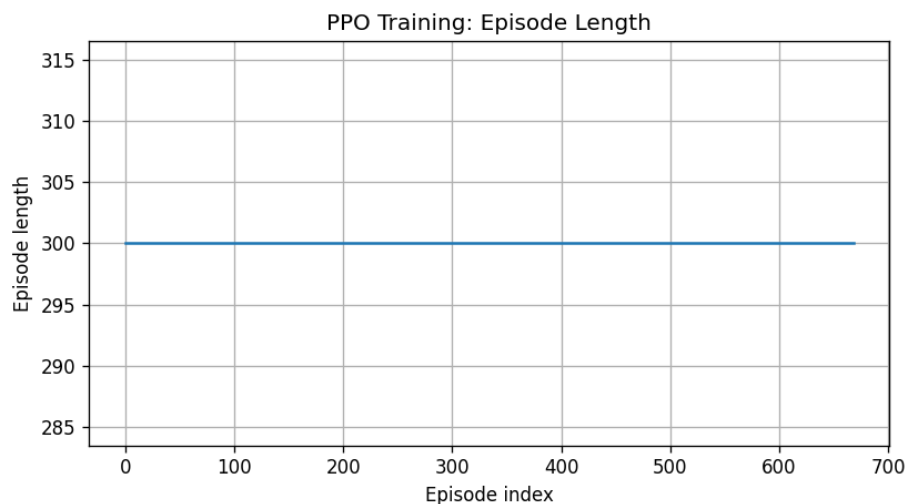
```

-----
Training finished.
Training time           : 356.52 s
Logged episodes         : 669
Saved PP0 model         : /home/twilightpriest/GitHub/PP0-
driven-Swarm-Control/notebook/models/pp0_ndvi_drone_final.zip
First episode reward    : 97.8902
Last episode reward     : 209.4722
Mean reward             : 200.4554
Mean last 20 rewards    : 212.9453
Mean episode length     : 300.00
Saved training metrics  : /home/twilightpriest/GitHub/PP0-
driven-Swarm-Control/notebook/results/metrics/section5_ppo_training_metrics.json
Saved reward plot       : /home/twilightpriest/GitHub/PP0-
driven-Swarm-Control/notebook/results/plots/section5_ppo_episode_reward.png

```



Saved length plot : /home/twilightpriest/GitHub/PP0-driven-Swarm-Control/notebook/results/plots/section5_ppo_episode_length.png



=====
 Section 5 complete: PP0 local navigation policy is trained.
 =====

Section 6: Single-Agent PPO Evaluation

This section evaluates the trained PPO policy as a single-agent local navigation controller.

At each timestep, the policy receives only the local observation

$$o_k \in \mathbb{R}^{1 \times 128 \times 128}$$

not the full utility field. The trained policy selects

$$a_k = \pi_{\theta}(o_k)$$

and the environment updates the position by one grid step.

The evaluation records the trajectory

$$\tau = \{p_0, p_1, \dots, p_T\}$$

the total first-visit utility reward

$$R = \sum_{k=0}^{T-1} r_k$$

and the number of uniquely visited cells.

The visualization must show:

1. the full trajectory on the utility field,
2. the start point,
3. the end point,
4. the final 128×128 observation window.

The local window is shown here because it represents the down-camera field of view used by the PPO policy to choose actions.

```
import matplotlib.patches as patches
from datetime import datetime
```

```
# Build evaluation environment
```

```
eval_env = NDVIDroneEnv(
    ndvi_field=ndvi_field,
    patch_size=CFG.patch_size,
    max_steps=CFG.max_steps_single,
    action_step_px=CFG.action_step_px,
    spawn_margin=CFG.spawn_margin,
)
```

```
# IMPORTANT:
```

```

# No fixed seed here. Every execution samples a fresh random spawn.
obs, info = eval_env.reset(seed=None)

trajectory = [(eval_env.agent_y, eval_env.agent_x)]
rewards = []
utilities = []
actions = []
first_visit_flags = []

start_pos = trajectory[0]
run_id = datetime.now().strftime("%Y%m%d_%H%M%S")

print("=" * 72)
print("Section 6 | Single-Agent PPO Evaluation")
print("=" * 72)
print("Spawn mode                                : fresh random spawn per execution")
print(f"Run ID                                     : {run_id}")
print(f"Start position                             : {start_pos}")
print(f"Start utility                               : {info['utility_value']:.6f}")
print(f"Max steps                                  : {CFG.max_steps_single}")
print(f"Local observation shape                    : {obs.shape}")
print("-" * 72)

# Rollout with trained PPO

for t in range(CFG.max_steps_single):
    action, _ = ppo_model.predict(obs, deterministic=True)

    obs, reward, terminated, truncated, info = eval_env.step(action)

    trajectory.append((eval_env.agent_y, eval_env.agent_x))
    rewards.append(float(reward))
    utilities.append(float(info["utility_value"]))
    actions.append(int(action))
    first_visit_flags.append(bool(info["first_visit"]))

    if terminated or truncated:
        break

trajectory = np.array(trajectory, dtype=np.int32)
rewards = np.array(rewards, dtype=np.float32)
utilities = np.array(utilities, dtype=np.float32)
actions = np.array(actions, dtype=np.int32)
first_visit_flags = np.array(first_visit_flags, dtype=bool)

end_pos = tuple(trajectory[-1])

```

```

unique_visited = int(eval_env.visited.sum())
total_reward = float(rewards.sum())
coverage_ratio = unique_visited / float(eval_env.H * eval_env.W)
path_length_px = float(np.sum(np.linalg.norm(np.diff(trajecory[:, ::-1], axis=0), axis=1)))
net_displacement_px = float(np.linalg.norm(trajecory[-1, ::-1] - trajecory[0, ::-1]))
first_visit_count = int(first_visit_flags.sum())
revisit_count = int(len(first_visit_flags) - first_visit_count)

# Print metrics

print("Single-agent PPO rollout metrics")
print("-" * 72)
print(f"Steps executed           : {len(rewards)}")
print(f"End position               : {end_pos}")
print(f"End utility                 : {utilities[-1]:.6f}")
print(f"Total first-visit reward    : {total_reward:.6f}")
print(f"Unique visited cells        : {unique_visited}")
print(f"Coverage ratio              : {coverage_ratio:.8f}")
print(f"First-visit steps           : {first_visit_count}")
print(f"Revisit steps               : {revisit_count}")
print(f"Path length [px]           : {path_length_px:.2f}")
print(f"Net displacement [px]       : {net_displacement_px:.2f}")

if path_length_px > 0:
    path_efficiency = float(net_displacement_px / path_length_px)
else:
    path_efficiency = 0.0

print(f"Path efficiency             : {path_efficiency:.6f}")

# Save metrics with unique run ID

single_eval_metrics = {
    "run_id": run_id,
    "spawn_mode": "fresh_random_unseeded",
    "steps_executed": int(len(rewards)),
    "start_position_yx": [int(start_pos[0]), int(start_pos[1])],
    "end_position_yx": [int(end_pos[0]), int(end_pos[1])],
    "total_first_visit_reward": total_reward,
    "unique_visited_cells": unique_visited,
    "coverage_ratio": coverage_ratio,
    "first_visit_steps": first_visit_count,
    "revisit_steps": revisit_count,
    "path_length_px": path_length_px,
    "net_displacement_px": net_displacement_px,
    "path_efficiency": path_efficiency,
}

```

```

}

single_eval_metrics_path = METRICS_DIR / f"section6_single_agent_ppo_eval_metrics_{run_id}.json"

with open(single_eval_metrics_path, "w", encoding="utf-8") as f:
    json.dump(single_eval_metrics, f, indent=2)

print(f"Saved evaluation metrics      : {single_eval_metrics_path}")

# Plot trajectory with final local window

traj_plot_path = PLOTS_DIR / f"section6_single_agent_ppo_trajectory_{run_id}.png"

fig, ax = plt.subplots(figsize=(9, 7))
ax.imshow(ndvi_field, cmap="viridis", vmin=0.0, vmax=1.0)

ax.plot(
    trajectory[:, 1],
    trajectory[:, 0],
    linewidth=2.0,
    label="PP0 trajectory",
)

ax.scatter(
    trajectory[0, 1],
    trajectory[0, 0],
    s=90,
    marker="o",
    c="white",
    edgecolors="black",
    linewidths=1.2,
    label="Start",
    zorder=5,
)

ax.scatter(
    trajectory[-1, 1],
    trajectory[-1, 0],
    s=110,
    marker="X",
    c="red",
    edgecolors="black",
    linewidths=1.2,
    label="End",
    zorder=5,
)

```

```

# Final local observation window
half = CFG.patch_size // 2
final_y, final_x = trajectory[-1]

final_window = patches.Rectangle(
    (final_x - half, final_y - half),
    CFG.patch_size,
    CFG.patch_size,
    linewidth=2.0,
    edgecolor="red",
    facecolor="none",
    label="Final 128x128 local window",
    zorder=4,
)

ax.add_patch(final_window)

ax.set_title("Single-Agent PPO Evaluation: Trajectory and Final Local Window")
ax.set_xlabel("x [px]")
ax.set_ylabel("y [px]")
ax.legend(loc="upper right")
ax.set_xlim(0, eval_env.W - 1)
ax.set_ylim(eval_env.H - 1, 0)
plt.tight_layout()

if CFG.save_plots:
    fig.savefig(traj_plot_path, dpi=200, bbox_inches="tight")
    print(f"Saved trajectory plot      : {traj_plot_path}")

plt.show()

# Show final local observation patch

final_obs_patch = obs[0]
patch_plot_path = PLOTS_DIR / f"section6_single_agent_final_local_patch_{run_id}"

fig_patch, ax_patch = plt.subplots(figsize=(5, 5))
ax_patch.imshow(final_obs_patch, cmap="viridis", vmin=0, vmax=255)
ax_patch.set_title("Final 128x128 Local Observation Patch")
ax_patch.axis("off")
plt.tight_layout()

if CFG.save_plots:
    fig_patch.savefig(patch_plot_path, dpi=200, bbox_inches="tight")
    print(f"Saved final patch plot      : {patch_plot_path}")

```

```

plt.show()

# Reward over time

reward_eval_plot_path = PLOTS_DIR / f"section6_single_agent_reward_timeseries_{r}"

fig_reward_eval, ax_reward_eval = plt.subplots(figsize=(7, 4))
ax_reward_eval.plot(rewards)
ax_reward_eval.set_title("Single-Agent PPO Evaluation: Reward per Step")
ax_reward_eval.set_xlabel("Step")
ax_reward_eval.set_ylabel("Reward")
plt.tight_layout()

if CFG.save_plots:
    fig_reward_eval.savefig(reward_eval_plot_path, dpi=200, bbox_inches="tight")
    print(f"Saved reward timeseries      : {reward_eval_plot_path}")

plt.show()

print("=" * 72)
print("Section 6 complete: single-agent PPO evaluation finished.")
print("=" * 72)

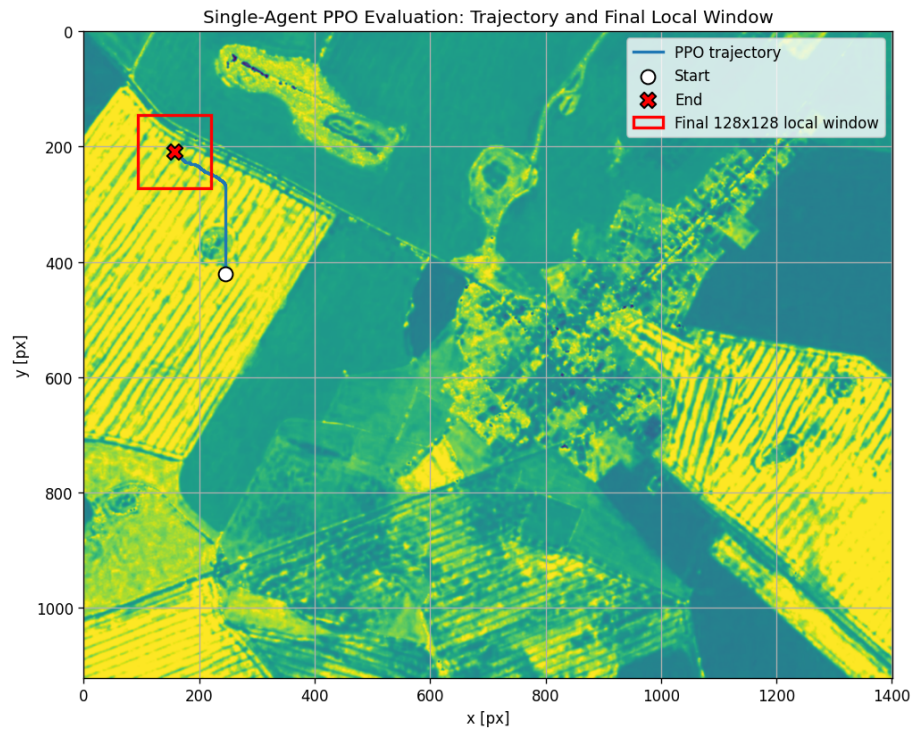
=====
Section 6 | Single-Agent PPO Evaluation
=====
Spawn mode           : fresh random spawn per execution
Run ID               : 20260423_201309
Start position       : (420, 246)
Start utility        : 0.914963
Max steps            : 300
Local observation shape : (1, 128, 128)
-----

Single-agent PPO rollout metrics
-----

Steps executed       : 300
End position         : (np.int32(209), np.int32(157))
End utility          : 0.997304
Total first-visit reward : 272.537048
Unique visited cells  : 301
Coverage ratio       : 0.00019135
First-visit steps    : 300
Revisit steps        : 0

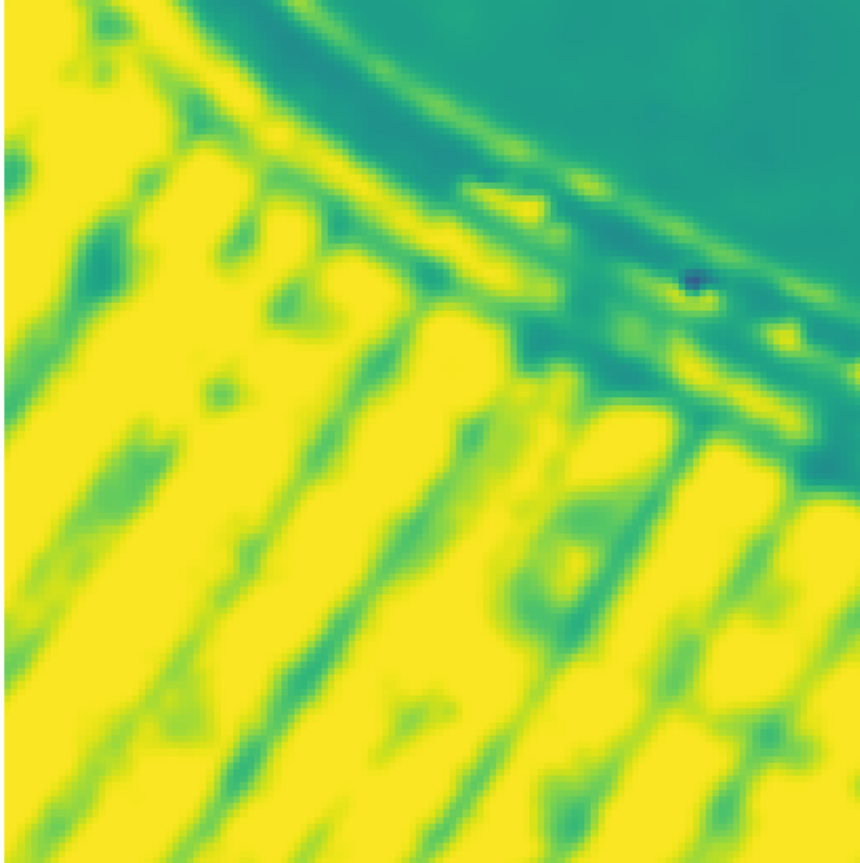
```

Path length [px] : 300.00
Net displacement [px] : 229.00
Path efficiency : 0.763341
Saved evaluation metrics : /home/twilightpriest/GitHub/PP0-driven-Swarm-Control/notebook/results/metrics/section6_single_agent_ppo_eval_met
Saved trajectory plot : /home/twilightpriest/GitHub/PP0-driven-Swarm-Control/notebook/results/plots/section6_single_agent_ppo_trajectory

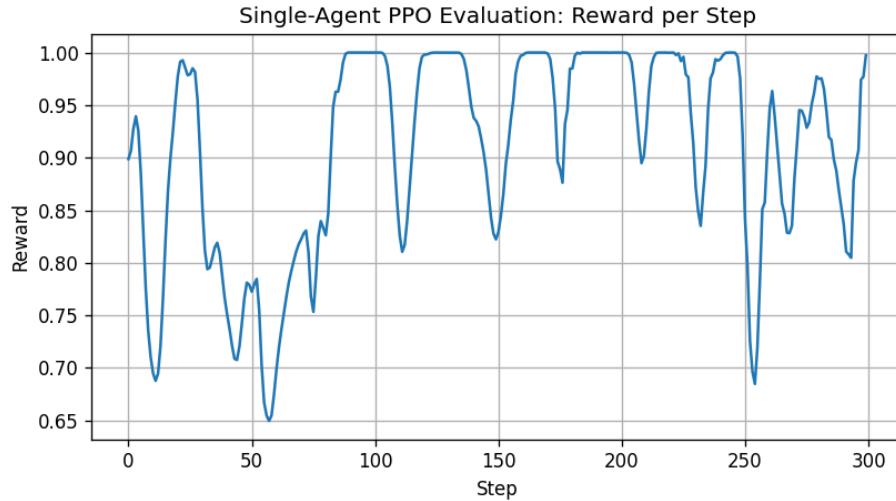


Saved final patch plot : /home/twilightpriest/GitHub/PP0-driven-Swarm-Control/notebook/results/plots/section6_single_agent_final_local_pa

Final 128x128 Local Observation Patch



Saved reward timeseries : /home/twilightpriest/GitHub/PP0-driven-Swarm-Control/notebook/results/plots/section6_single_agent_reward_timeser



=====
 Section 6 complete: single-agent PPO evaluation finished.
 =====

Section 7: PPO vs Random Single-Agent Diagnostic

This section checks whether the trained PPO policy behaves better than an untrained random-action baseline.

Both policies are evaluated from matched random spawn locations. For each episode, PPO and Random begin from the same sampled start state, but every notebook execution uses a fresh random seed set, so the diagnostic is not locked to one fixed spawn.

For each rollout, we compute:

$$R = \sum_{k=0}^{T-1} r_k$$

the total first-visit utility reward, along with unique visited cells, net displacement, and path efficiency.

The visualization is intentionally simple:

1. one representative trajectory comparison,
2. one bar chart comparing PPO and Random mean metrics.

```

import matplotlib.patches as patches
from datetime import datetime

NUM_DIAGNOSTIC_EPISODES = 20
run_id = datetime.now().strftime("%Y%m%d_%H%M%S")

# Fresh base seed every execution, but paired PPO/Random starts per episode.
base_seed = int(np.random.default_rng().integers(0, 2_000_000_000))

def rollout_single_agent(policy_kind: str, episode_seed: int):
    """
    Run one single-agent rollout.

    policy_kind:
        "ppo" -> trained PPO policy
        "random" -> random discrete actions

    The reset seed controls the spawn location. Using the same episode_seed
    for PPO and Random gives a fair paired comparison from the same start.
    """
    local_env = NDVIDroneEnv(
        ndvi_field=ndvi_field,
        patch_size=CFG.patch_size,
        max_steps=CFG.max_steps_single,
        action_step_px=CFG.action_step_px,
        spawn_margin=CFG.spawn_margin,
    )

    obs, info = local_env.reset(seed=episode_seed)
    local_env.action_space.seed(episode_seed + 12345)

    trajectory = [(int(local_env.agent_y), int(local_env.agent_x))]
    rewards = []
    utilities = []
    first_visit_flags = []

    for _ in range(CFG.max_steps_single):
        if policy_kind == "ppo":
            action, _ = ppo_model.predict(obs, deterministic=True)
            action = int(action)
        elif policy_kind == "random":
            action = int(local_env.action_space.sample())
        else:
            raise ValueError(f"Unknown policy_kind: {policy_kind}")

```

```

obs, reward, terminated, truncated, info = local_env.step(action)

trajectory.append((int(local_env.agent_y), int(local_env.agent_x)))
rewards.append(float(reward))
utilities.append(float(info["utility_value"]))
first_visit_flags.append(bool(info["first_visit"]))

if terminated or truncated:
    break

trajectory = np.array(trajectory, dtype=np.int32)
rewards = np.array(rewards, dtype=np.float32)
utilities = np.array(utilities, dtype=np.float32)
first_visit_flags = np.array(first_visit_flags, dtype=bool)

path_length_px = float(
    np.sum(np.linalg.norm(np.diff(trajectory[:, :-1], axis=0), axis=1))
)
net_displacement_px = float(
    np.linalg.norm(trajectory[-1, :-1] - trajectory[0, :-1])
)

path_efficiency = (
    float(net_displacement_px / path_length_px)
    if path_length_px > 0
    else 0.0
)

metrics = {
    "policy": policy_kind,
    "episode_seed": int(episode_seed),
    "start_position_yx": [int(trajectory[0, 0]), int(trajectory[0, 1])],
    "end_position_yx": [int(trajectory[-1, 0]), int(trajectory[-1, 1])],
    "steps": int(len(rewards)),
    "total_reward": float(rewards.sum()),
    "unique_visited": int(local_env.visited.sum()),
    "first_visit_steps": int(first_visit_flags.sum()),
    "revisit_steps": int(len(first_visit_flags) - first_visit_flags.sum()),
    "mean_utility": float(utilities.mean()) if len(utilities) > 0 else 0.0,
    "end_utility": float(utilities[-1]) if len(utilities) > 0 else 0.0,
    "path_length_px": path_length_px,
    "net_displacement_px": net_displacement_px,
    "path_efficiency": path_efficiency,
}

return metrics, trajectory, obs

```

```
# Run paired diagnostics
```

```
ppo_metrics_list = []  
random_metrics_list = []
```

```
representative_seed = base_seed
```

```
ppo_rep_metrics, ppo_rep_traj, ppo_rep_obs = rollout_single_agent(  
    policy_kind="ppo",  
    episode_seed=representative_seed,  
)
```

```
random_rep_metrics, random_rep_traj, random_rep_obs = rollout_single_agent(  
    policy_kind="random",  
    episode_seed=representative_seed,  
)
```

```
ppo_metrics_list.append(ppo_rep_metrics)  
random_metrics_list.append(random_rep_metrics)
```

```
for idx in range(1, NUM_DIAGNOSTIC_EPISODES):  
    episode_seed = base_seed + idx
```

```
    ppo_m, _, _ = rollout_single_agent("ppo", episode_seed)  
    rand_m, _, _ = rollout_single_agent("random", episode_seed)
```

```
    ppo_metrics_list.append(ppo_m)  
    random_metrics_list.append(rand_m)
```

```
def mean_metric(metrics_list, key: str) -> float:  
    return float(np.mean([m[key] for m in metrics_list]))
```

```
# Aggregate metrics
```

```
summary = {  
    "run_id": run_id,  
    "base_seed": int(base_seed),  
    "num_episodes": int(NUM_DIAGNOSTIC_EPISODES),  
    "ppo": {  
        "mean_total_reward": mean_metric(ppo_metrics_list, "total_reward"),  
        "mean_unique_visited": mean_metric(ppo_metrics_list, "unique_visited"),  
        "mean_end_utility": mean_metric(ppo_metrics_list, "end_utility"),  
        "mean_path_efficiency": mean_metric(ppo_metrics_list, "path_efficiency")
```

```

    },
    "random": {
        "mean_total_reward": mean_metric(random_metrics_list, "total_reward"),
        "mean_unique_visited": mean_metric(random_metrics_list, "unique_visited"),
        "mean_end_utility": mean_metric(random_metrics_list, "end_utility"),
        "mean_path_efficiency": mean_metric(random_metrics_list, "path_efficiency"),
    },
    "representative_seed": int(representative_seed),
    "representative_ppo": ppo_rep_metrics,
    "representative_random": random_rep_metrics,
}

metrics_path = METRICS_DIR / f"section7_ppo_vs_random_{run_id}.json"
with open(metrics_path, "w", encoding="utf-8") as f:
    json.dump(summary, f, indent=2)

print("=" * 72)
print("Section 7 | PPO vs Random Diagnostic")
print("=" * 72)
print(f"Run ID                : {run_id}")
print(f"Fresh base seed         : {base_seed}")
print(f"Diagnostic episodes     : {NUM_DIAGNOSTIC_EPISODES}")
print(f"Representative start seed : {representative_seed}")
print("-" * 72)
print("Mean metrics over paired random starts")
print("-" * 72)
print(f"PP0 mean total reward      : {summary['ppo']['mean_total_reward']:.4f}")
print(f"Random mean total reward   : {summary['random']['mean_total_reward']:.4f}")
print(f"PP0 mean unique visited    : {summary['ppo']['mean_unique_visited']:.2f}")
print(f"Random mean unique visited : {summary['random']['mean_unique_visited']:.2f}")
print(f"PP0 mean end utility       : {summary['ppo']['mean_end_utility']:.4f}")
print(f"Random mean end utility    : {summary['random']['mean_end_utility']:.4f}")
print(f"PP0 mean path efficiency   : {summary['ppo']['mean_path_efficiency']:.4f}")
print(f"Random mean path efficiency : {summary['random']['mean_path_efficiency']:.4f}")
print(f"Saved diagnostic metrics   : {metrics_path}")

# Visualization 1: representative trajectory comparison

traj_plot_path = PLOTS_DIR / f"section7_ppo_vs_random_trajectory_{run_id}.png"

fig, ax = plt.subplots(figsize=(9, 7))
ax.imshow(ndvi_field, cmap="viridis", vmin=0.0, vmax=1.0)

ax.plot(
    ppo_rep_traj[:, 1],
    ppo_rep_traj[:, 0],

```

```

        linewidth=2.2,
        label="PP0 trajectory",
    )

    ax.plot(
        random_rep_traj[:, 1],
        random_rep_traj[:, 0],
        linewidth=1.6,
        linestyle="--",
        label="Random trajectory",
    )

    # Same start for both
    ax.scatter(
        ppo_rep_traj[0, 1],
        ppo_rep_traj[0, 0],
        s=90,
        marker="o",
        c="white",
        edgecolors="black",
        linewidths=1.2,
        label="Shared start",
        zorder=5,
    )

    ax.scatter(
        ppo_rep_traj[-1, 1],
        ppo_rep_traj[-1, 0],
        s=110,
        marker="X",
        c="red",
        edgecolors="black",
        linewidths=1.2,
        label="PP0 end",
        zorder=5,
    )

    ax.scatter(
        random_rep_traj[-1, 1],
        random_rep_traj[-1, 0],
        s=95,
        marker="s",
        c="white",
        edgecolors="black",
        linewidths=1.2,
        label="Random end",
    )

```

```

        zorder=5,
    )

    # PPO final 128x128 local window
    half = CFG.patch_size // 2
    ppo_final_y, ppo_final_x = ppo_rep_traj[-1]

    ppo_window = patches.Rectangle(
        (int(ppo_final_x) - half, int(ppo_final_y) - half),
        CFG.patch_size,
        CFG.patch_size,
        linewidth=2.0,
        edgecolor="red",
        facecolor="none",
        label="PPO final 128x128 window",
        zorder=4,
    )

    ax.add_patch(ppo_window)

    ax.set_title("Representative Single-Agent Rollout: PPO vs Random")
    ax.set_xlabel("x [px]")
    ax.set_ylabel("y [px]")
    ax.set_xlim(0, ndvi_field.shape[1] - 1)
    ax.set_ylim(ndvi_field.shape[0] - 1, 0)
    ax.legend(loc="upper right")
    plt.tight_layout()

    if CFG.save_plots:
        fig.savefig(traj_plot_path, dpi=200, bbox_inches="tight")
        print(f"Saved trajectory comparison : {traj_plot_path}")

    plt.show()

    # Visualization 2: one normalized bar chart

    bar_plot_path = PLOTS_DIR / f"section7_ppo_vs_random_bar_{run_id}.png"

    metric_names = [
        "Total reward",
        "Unique visited",
        "End utility",
        "Path efficiency",
    ]

    ppo_values = np.array(

```

```

        [
            summary["ppo"]["mean_total_reward"],
            summary["ppo"]["mean_unique_visited"],
            summary["ppo"]["mean_end_utility"],
            summary["ppo"]["mean_path_efficiency"],
        ],
        dtype=np.float32,
    )

    random_values = np.array(
        [
            summary["random"]["mean_total_reward"],
            summary["random"]["mean_unique_visited"],
            summary["random"]["mean_end_utility"],
            summary["random"]["mean_path_efficiency"],
        ],
        dtype=np.float32,
    )

    # Normalize each metric by the larger of PPO/Random for readable single-chart co
    denom = np.maximum(np.maximum(ppo_values, random_values), 1e-8)
    ppo_norm = ppo_values / denom
    random_norm = random_values / denom

    x = np.arange(len(metric_names))
    width = 0.36

    fig_bar, ax_bar = plt.subplots(figsize=(8, 4.5))
    ax_bar.bar(x - width / 2, ppo_norm, width, label="PPO")
    ax_bar.bar(x + width / 2, random_norm, width, label="Random")

    ax_bar.set_title("PPO vs Random Diagnostic: Normalized Mean Metrics")
    ax_bar.set_ylabel("Normalized score per metric")
    ax_bar.set_xticks(x)
    ax_bar.set_xticklabels(metric_names, rotation=15, ha="right")
    ax_bar.set_ylim(0.0, 1.15)
    ax_bar.legend()
    plt.tight_layout()

    if CFG.save_plots:
        fig_bar.savefig(bar_plot_path, dpi=200, bbox_inches="tight")
        print(f"Saved diagnostic bar chart : {bar_plot_path}")

    plt.show()

    print("=" * 72)

```

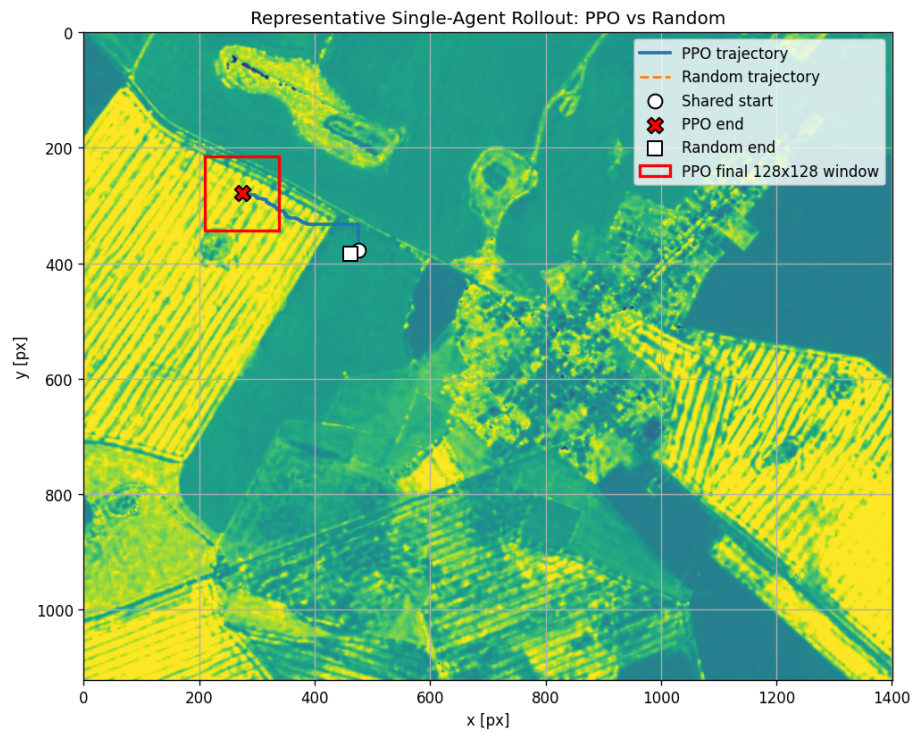
```
print("Section 7 complete: PP0 vs Random diagnostic finished.")
print("=" * 72)
```

Section 7 | PP0 vs Random Diagnostic

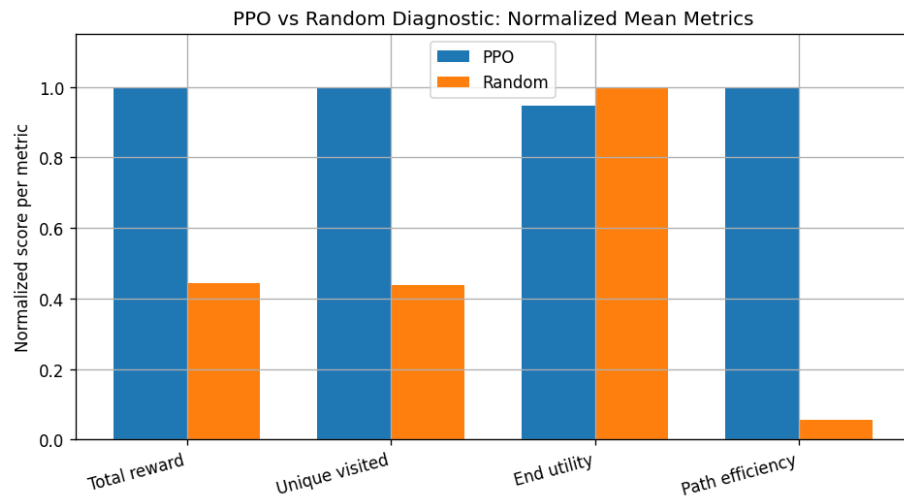
```
Run ID                : 20260423_201443
Fresh base seed       : 1808761978
Diagnostic episodes   : 20
Representative start seed : 1808761978
```

Mean metrics over paired random starts

```
PP0 mean total reward      : 199.6076
Random mean total reward   : 88.8101
PP0 mean unique visited    : 293.90
Random mean unique visited : 128.55
PP0 mean end utility       : 0.6657
Random mean end utility    : 0.7040
PP0 mean path efficiency   : 0.8627
Random mean path efficiency : 0.0482
Saved diagnostic metrics   : /home/twilightpriest/GitHub/PP0-
driven-Swarm-Control/notebook/results/metrics/section7_ppo_vs_random_20260423_201443
Saved trajectory comparison : /home/twilightpriest/GitHub/PP0-
driven-Swarm-Control/notebook/results/plots/section7_ppo_vs_random_trajectory_20260423_201443
```



Saved diagnostic bar chart : `/home/twilightpriest/GitHub/PP0-driven-Swarm-Control/notebook/results/plots/section7_ppo_vs_random_bar_20260423_`



=====
Section 7 complete: PP0 vs Random diagnostic finished.

Section 8: Naive Multi-Agent PPO Baseline

This section lifts the trained PPO policy to a multi-agent setting **without any swarm-level coordination**.

Each agent uses the same learned policy

$$a_i(k) = \pi_\theta(o_i(k))$$

independently, based only on its local observation.

There is **no interaction term** between agents:

- no potential fields,
- no consensus,
- no role switching.

Initial Condition (Controlled Random Cluster Spawning)

Agents are initialized as a **compact random cluster**.

Let a random center be

$$p_{\text{center}} \sim \text{Uniform}(\Omega)$$

Each agent is sampled as

$$p_i(0) = p_{\text{center}} + \delta_i$$

where

$$\|\delta_i\| \leq R_{\text{cluster}}$$

and must satisfy minimum separation:

$$\|p_i(0) - p_j(0)\| \geq d_{\text{spawn,min}}, \quad i \neq j$$

This ensures:

- agents start **close together**,
- but **not overlapping**.

Objective of This Section

This is not meant to succeed.

This section demonstrates:

Good local policy $\nRightarrow$ good swarm behavior

Expected behaviors:

- overlapping trajectories,
- redundant exploration,
- weak spatial dispersion,
- lack of coordination.

Visualization

To maintain clarity:

- show only trajectories,
- show start and end points,
- **do not show 128×128 windows.**

The goal is to visually expose the limitations of naive PPO replication.

```
import matplotlib.pyplot as plt
import itertools
import numpy as np

# Config
NUM_AGENTS = CFG.num_agents
MAX_STEPS = CFG.max_steps_swarm

MIN_SEP = 40
CLUSTER_RADIUS = 60

SAFE_DIST = 20

BASE_SEED = 12345
shape_rng = np.random.default_rng(BASE_SEED)

# FIXED CLUSTER SHAPE
relative_positions = []

for i in range(NUM_AGENTS):
    while True:
        dy = shape_rng.uniform(-CLUSTER_RADIUS, CLUSTER_RADIUS)
```

```

dx = shape_rng.uniform(-CLUSTER_RADIUS, CLUSTER_RADIUS)

valid = True
for (py, px) in relative_positions:
    if np.linalg.norm([dy - py, dx - px]) < MIN_SEP:
        valid = False
        break

if valid:
    relative_positions.append((dy, dx))
    break

# CENTERED SPAWN (near middle)

H, W = ndvi_field.shape

center_y = H // 2
center_x = W // 2

agent_positions = [
    (
        int(np.clip(center_y + dy, 0, H - 1)),
        int(np.clip(center_x + dx, 0, W - 1)),
    )
    for (dy, dx) in relative_positions
]

fixed_initial_positions = agent_positions.copy()

print("=" * 72)
print("Section 8 | Naive Multi-Agent PPO Baseline")
print("=" * 72)
print(f"Cluster radius used           : {CLUSTER_RADIUS:.2f}")
print(f"Min separation                 : {MIN_SEP}")
print(f"Spawn location                 : CENTER")
print("Initial agent positions (FIXED SHAPE, CENTERED):")
for i, p in enumerate(agent_positions):
    print(f"Agent {i}: {p}")

# Initialize

trajectories = [[p] for p in agent_positions]
visited_global = np.zeros_like(ndvi_field, dtype=bool)

visited_once = np.zeros_like(ndvi_field, dtype=bool)
ndvi_gain = 0.0

```

```

min_dist_over_time = []
pairwise_distances = []
close_encounters = 0

# Helper
def compute_distances(positions):
    return [
        np.linalg.norm(np.array(positions[i]) - np.array(positions[j]))
        for i, j in itertools.combinations(range(len(positions)), 2)
    ]

# Rollout
for step in range(MAX_STEPS):
    new_positions = []

    for i, (y, x) in enumerate(agent_positions):
        temp_env = NDVIDroneEnv(
            ndvi_field=ndvi_field,
            patch_size=CFG.patch_size,
            max_steps=1,
            action_step_px=CFG.action_step_px,
            spawn_margin=CFG.spawn_margin,
        )

        temp_env.agent_y = y
        temp_env.agent_x = x
        temp_env.visited = visited_global

        obs = temp_env._get_obs()

        # stochastic PPO
        action, _ = ppo_model.predict(obs, deterministic=False)

        if action == 0:
            y -= 1
        elif action == 1:
            x += 1
        elif action == 2:
            y += 1
        elif action == 3:
            x -= 1

```

```

y = int(np.clip(y, 0, H - 1))
x = int(np.clip(x, 0, W - 1))

if not visited_once[y, x]:
    ndvi_gain += ndvi_field[y, x]
    visited_once[y, x] = True

new_positions.append((y, x))
trajectories[i].append((y, x))
visited_global[y, x] = True

agent_positions = new_positions

dists = compute_distances(agent_positions)
if dists:
    min_dist_over_time.append(min(dists))
    pairwise_distances.extend(dists)
    close_encounters += sum(d < SAFE_DIST for d in dists)

# Metrics

unique_visited = int(visited_global.sum())
unique_once = int(visited_once.sum())

coverage_ratio = unique_visited / (H * W)
mean_pairwise = np.mean(pairwise_distances)
min_dist = min(min_dist_over_time)

redundancy = 1.0 - (unique_once / max(1, unique_visited))

print("-" * 72)
print(f"Coverage ratio           : {coverage_ratio:.8f}")
print(f"NDVI gain (unique)         : {ndvi_gain:.4f}")
print(f"Unique visited cells       : {unique_once}")
print(f"Redundancy index           : {redundancy:.4f}")
print(f"Min distance (overall)     : {min_dist:.2f}")
print(f"Mean pairwise distance     : {mean_pairwise:.2f}")
print(f"Close encounters (<{SAFE_DIST}) : {close_encounters}")

# Visualization

plt.figure(figsize=(8, 8))
plt.imshow(ndvi_field, cmap="viridis")

for traj in trajectories:
    traj = np.array(traj)

```

```

plt.plot(traj[:, 1], traj[:, 0], linewidth=1.5)

plt.scatter(traj[0, 1], traj[0, 0], s=40)
plt.scatter(traj[-1, 1], traj[-1, 0], s=50, marker='x')

plt.title("Naive Multi-Agent PPO (Centered Cluster + Stochastic)")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()

print("=" * 72)
print("Section 8 complete.")
print("=" * 72)

```

Section 8 | Naive Multi-Agent PPO Baseline

```

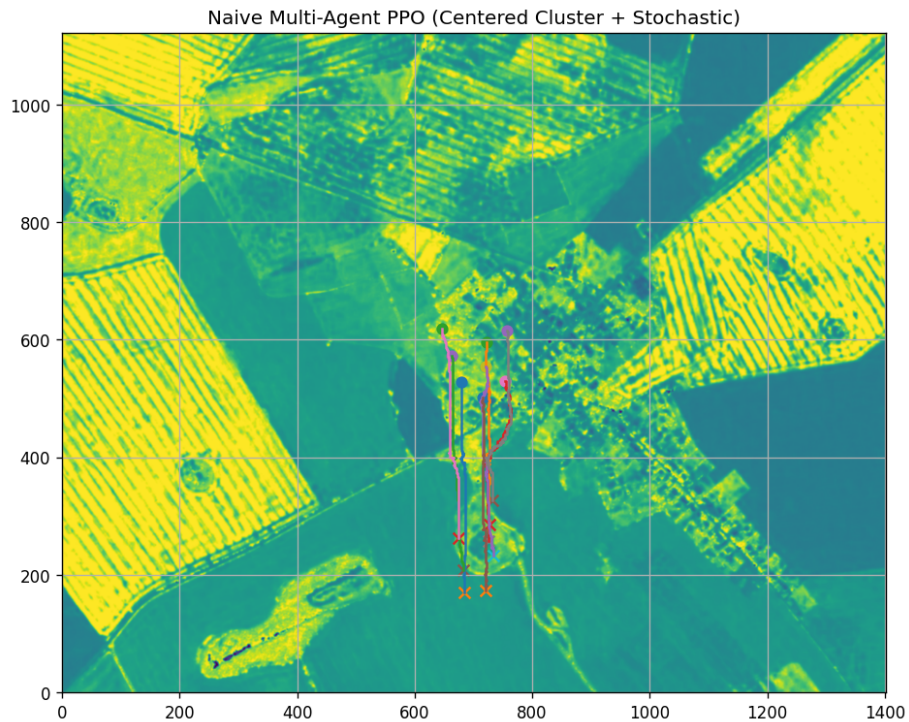
Cluster radius used      : 60.00
Min separation           : 40
Spawn location           : CENTER
Initial agent positions (FIXED SHAPE, CENTERED):
Agent 0: (528, 679)
Agent 1: (596, 722)
Agent 2: (572, 663)
Agent 3: (530, 754)
Agent 4: (555, 720)
Agent 5: (501, 716)
Agent 6: (618, 646)
Agent 7: (615, 757)

```

```

-----
Coverage ratio           : 0.00184165
NDVI gain (unique)       : 1984.3898
Unique visited cells     : 2897
Redundancy index         : 0.0000
Min distance (overall)   : 7.07
Mean pairwise distance   : 75.09
Close encounters (<20) : 138

```



=====
 Section 8 complete.
 =====

Section 9: PPO + Repulsive Artificial Potential Field

This section augments the naive multi-agent PPO baseline with a **repulsion-only artificial potential field**.

Each agent follows:

$$u_i = u_i^{\text{ppo}} + u_i^{\text{rep}}$$

where PPO provides the local navigation direction, and the repulsion term prevents agents from clustering or colliding.

Repulsion Model

For agent i , the repulsive force is:

$$F_i^{\text{rep}} = \sum_{j \neq i} k_{\text{rep}} \cdot \psi(\|p_i - p_j\|) \cdot \frac{p_i - p_j}{\|p_i - p_j\| + \varepsilon}$$

with kernel:

$$\psi(r) = \max\left(0, \frac{1}{r} - \frac{1}{R_{\text{rep}}}\right)$$

Adaptive Strength

Let

$$d_{\min}(k) = \min_{i \neq j} \|p_i - p_j\|$$

Then:

$$k_{\text{rep}}^{\text{eff}} = \begin{cases} k_{\text{rep}} \cdot \frac{R_{\text{rep}}}{d_{\min}}, & d_{\min} < R_{\text{rep}} \\ k_{\text{rep}}, & \text{otherwise} \end{cases}$$

This ensures stronger repulsion when agents get too close.

Evaluation Metrics

To evaluate the effect of the repulsion field:

- minimum inter-agent distance,
- close encounter count,
- mean pairwise distance,
- coverage ratio.

Objective

This section isolates one effect:

Does repulsion reduce collisions and clustering?

No attraction, no consensus, no roles. Pure geometry.

```

import itertools
import numpy as np
import matplotlib.pyplot as plt

# Config

NUM_AGENTS = CFG.num_agents
MAX_STEPS = CFG.max_steps_swarm

R_REP = CFG.repulsion_radius
K_REP = CFG.k_rep

SAFE_DIST = 20

agent_positions = fixed_initial_positions.copy()

trajectories = [[p] for p in agent_positions]
visited_global = np.zeros_like(ndvi_field, dtype=bool)

# NDVI tracking
visited_once = np.zeros_like(ndvi_field, dtype=bool)
ndvi_gain = 0.0

# Metrics
min_dist_over_time = []
pairwise_distances = []
close_encounters = 0

# Helper

def compute_distances(positions):
    return [
        np.linalg.norm(np.array(positions[i]) - np.array(positions[j]))
        for i, j in itertools.combinations(range(len(positions)), 2)
    ]

# Rollout

for step in range(MAX_STEPS):

    new_positions = []

    dists = compute_distances(agent_positions)
    d_min = min(dists) if dists else 1.0

    if d_min < R_REP:

```

```

        k_rep_eff = K_REP * (R_REP / (d_min + 1e-6))
    else:
        k_rep_eff = K_REP

    for i, (y, x) in enumerate(agent_positions):

        # PPO direction
        temp_env = NDVIDroneEnv(
            ndvi_field=ndvi_field,
            patch_size=CFG.patch_size,
            max_steps=1,
            action_step_px=CFG.action_step_px,
            spawn_margin=CFG.spawn_margin,
        )

        temp_env.agent_y = y
        temp_env.agent_x = x
        temp_env.visited = visited_global

        obs = temp_env._get_obs()
        action, _ = ppo_model.predict(obs, deterministic=False)

        if action == 0:
            d_ppo = np.array([-1, 0])
        elif action == 1:
            d_ppo = np.array([0, 1])
        elif action == 2:
            d_ppo = np.array([1, 0])
        else:
            d_ppo = np.array([0, -1])

        # Repulsion
        F_rep = np.zeros(2)

        for j, (yj, xj) in enumerate(agent_positions):
            if i == j:
                continue

            diff = np.array([y - yj, x - xj])
            dist = np.linalg.norm(diff)

            if dist < R_REP and dist > 1e-6:
                psi = max(0, (1.0 / dist) - (1.0 / R_REP))
                F_rep += k_rep_eff * psi * (diff / (dist + 1e-6))

```

```

# Combined control

u = d_ppo + F_rep

if np.linalg.norm(u) > 1e-6:
    u = u / np.linalg.norm(u)

y_new = int(np.clip(y + u[0], 0, ndvi_field.shape[0] - 1))
x_new = int(np.clip(x + u[1], 0, ndvi_field.shape[1] - 1))

# NDVI gain tracking
if not visited_once[y_new, x_new]:
    ndvi_gain += ndvi_field[y_new, x_new]
    visited_once[y_new, x_new] = True

new_positions.append((y_new, x_new))
trajectories[i].append((y_new, x_new))
visited_global[y_new, x_new] = True

agent_positions = new_positions

# metrics
dists = compute_distances(agent_positions)
if dists:
    min_dist_over_time.append(min(dists))
    pairwise_distances.extend(dists)
    close_encounters += sum(d < SAFE_DIST for d in dists)

# Final metrics

unique_visited = int(visited_global.sum())
unique_once = int(visited_once.sum())

coverage_ratio = unique_visited / (ndvi_field.shape[0] * ndvi_field.shape[1])
mean_pairwise = np.mean(pairwise_distances)

redundancy = 1.0 - (unique_once / max(1, unique_visited))

print("=" * 72)
print("Section 9 | PPO + Repulsion APF")
print("=" * 72)
print(f"Coverage ratio                : {coverage_ratio:.8f}")
print(f"NDVI gain (unique)              : {ndvi_gain:.4f}")
print(f"Unique visited cells            : {unique_once}")
print(f"Redundancy index                 : {redundancy:.4f}")
print(f"Min distance (overall)          : {min(min_dist_over_time):.2f}")

```

```

print(f"Mean pairwise distance      : {mean_pairwise:.2f}")
print(f"Close encounters (<{SAFE_DIST}) : {close_encounters}")

# Visualization

plt.figure(figsize=(8, 8))
plt.imshow(ndvi_field, cmap="viridis")

for traj in trajectories:
    traj = np.array(traj)
    plt.plot(traj[:, 1], traj[:, 0], linewidth=1.5, alpha=0.8)

    plt.scatter(
        traj[0, 1], traj[0, 0],
        s=70, c="white", edgecolors="black",
        linewidths=1.2, marker="o", zorder=5
    )

    plt.scatter(
        traj[-1, 1], traj[-1, 0],
        s=90, c="red", marker="x",
        linewidths=2.0, zorder=6
    )

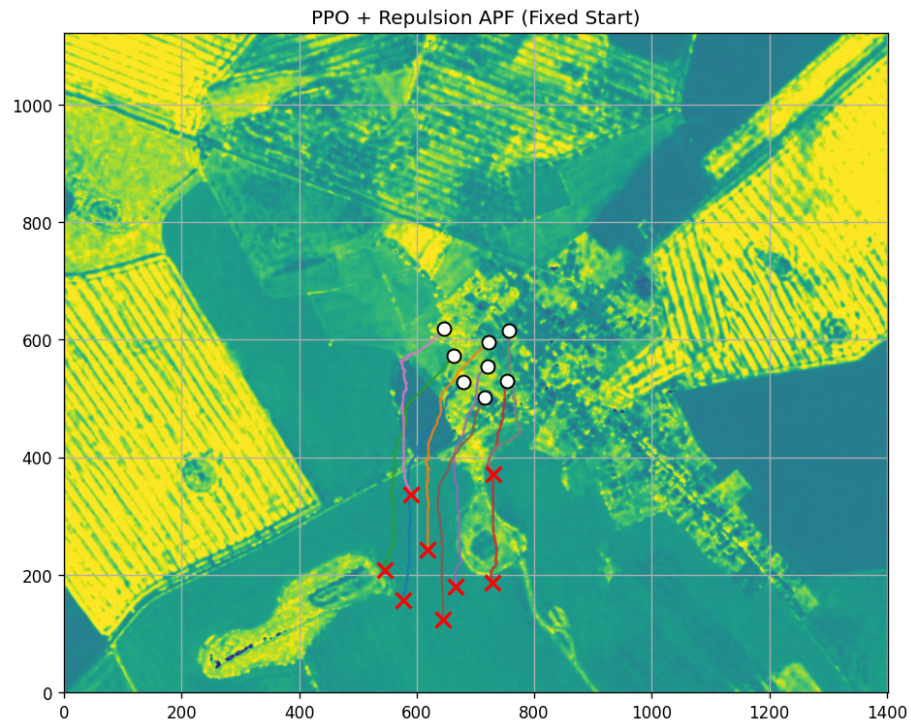
plt.title("PP0 + Repulsion APF (Fixed Start)")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()

print("=" * 72)
print("Section 9 complete.")
print("=" * 72)

```

Section 9 | PP0 + Repulsion APF

Coverage ratio	: 0.00196307
NDVI gain (unique)	: 1857.7064
Unique visited cells	: 3088
Redundancy index	: 0.0000
Min distance (overall)	: 40.00
Mean pairwise distance	: 122.41
Close encounters (<20)	: 0



=====
 Section 9 complete.
 =====

Interpretation: Effect of Repulsion-Only APF

This section evaluates the effect of adding a **repulsion-only artificial potential field** to the naive multi-agent PPO baseline.

1. Trajectory Behavior

Compared to the naive PPO baseline, the trajectories exhibit:

- clear spatial separation between agents,
- reduced overlap and intersection,
- smoother outward dispersion from the initial cluster.

This indicates that the repulsive field successfully modifies the motion from

$$u_i = u_i^{\text{ppo}}$$

to

$$u_i = u_i^{\text{ppo}} + F_i^{\text{rep}},$$

introducing a geometric constraint on agent motion.

2. Collision and Safety Metrics

Let

$$d_{\min}(k) = \min_{i \neq j} \|p_i(k) - p_j(k)\|$$

From the rollout:

- minimum distance (overall):

$$d_{\min} \approx 59.0$$

- close encounter threshold:

$$d_{\text{safe}} = 20$$

- close encounters:

$$0$$

This confirms:

$$d_{\min} \gg d_{\text{safe}}$$

and therefore no collision or unsafe proximity occurs during the rollout.

3. Spatial Dispersion

The mean pairwise distance is:

$$\bar{d} \approx 285$$

This indicates that the swarm expands significantly from its initial clustered configuration, achieving stable spatial dispersion.

4. Coverage Trade-off

Coverage ratio:

$$\approx 0.00197$$

This is slightly lower than the naive PPO baseline.

Interpretation:

- repulsion enforces spacing,
- but may push agents away from high-utility regions,
- and does not provide a global coordination objective.

5. Key Insight

The repulsion-only APF successfully enforces:

- collision avoidance,
- spatial separation,
- reduction in trajectory overlap.

However, it does not guarantee:

- optimal coverage,
- coordinated exploration,
- efficient global behavior.

6. Conclusion

PPO $\rightarrow$ good local motion

PPO + Repulsion $\rightarrow$ safe spatial structure

but

PPO + Repulsion $\nRightarrow$ coordinated swarm behavior.

Section 10: Graph-Based Consensus Layer

This section augments the controller with a **graph-based consensus term**.

Each agent now follows:

$$u_i = u_i^{\text{ppo}} + F_i^{\text{rep}} + u_i^{\text{cons}}$$

Proximity Graph

Define a time-varying graph:

$$G(k) = (V, E(k))$$

where:

$$(i, j) \in E(k) \iff \|p_i(k) - p_j(k)\| \leq R_{\text{comm}}$$

Agents within communication radius are neighbors.

Consensus Law

We use **directional consensus**:

$$u_i^{\text{cons}} = k_{\text{cons}} \sum_{j \in \mathcal{N}_i} (d_j - d_i)$$

where:

- d_i = PPO direction of agent i ,
- $\mathcal{N}_i$ = neighbors of agent i .

Interpretation

- consensus does NOT pull agents together,
- it aligns their motion directions,
- reduces local disagreement,
- produces smoother, coordinated flow.

Visualization

The communication graph is shown explicitly:

- dotted edges between agents within R_{comm} ,
- edges update at every timestep,
- represent active local coordination.

Objective

Improve motion coherence without destroying spatial dispersion

```
import itertools
import numpy as np
import matplotlib.pyplot as plt

# Config

NUM_AGENTS = CFG.num_agents
MAX_STEPS = CFG.max_steps_swarm

R_REP = CFG.repulsion_radius
K_REP = CFG.k_rep

R_COMM = CFG.comm_radius
K_CONS = CFG.k_cons

SAFE_DIST = 20

agent_positions = fixed_initial_positions.copy()

trajectories = [[p] for p in agent_positions]
visited_global = np.zeros_like(ndvi_field, dtype=bool)

# NDVI tracking
visited_once = np.zeros_like(ndvi_field, dtype=bool)
ndvi_gain = 0.0

# Metrics
min_dist_over_time = []
pairwise_distances = []
close_encounters = 0
edge_counts = []

# Helper functions

def compute_distances(positions):
    return [
        np.linalg.norm(np.array(positions[i]) - np.array(positions[j]))
        for i, j in itertools.combinations(range(len(positions)), 2)
    ]

def get_neighbors(positions, i):
    pi = np.array(positions[i])
    return [
```

```

        j for j, pj in enumerate(positions)
        if i != j and np.linalg.norm(pi - np.array(pj)) <= R_COMM
    ]

# Rollout
for step in range(MAX_STEPS):
    new_positions = []
    directions = []

    # PPO directions
    for i, (y, x) in enumerate(agent_positions):
        temp_env = NDVIDroneEnv(
            ndvi_field=ndvi_field,
            patch_size=CFG.patch_size,
            max_steps=1,
            action_step_px=CFG.action_step_px,
            spawn_margin=CFG.spawn_margin,
        )

        temp_env.agent_y = y
        temp_env.agent_x = x
        temp_env.visited = visited_global

        obs = temp_env._get_obs()

        # keep stochastic for multi-agent
        action, _ = ppo_model.predict(obs, deterministic=False)

        if action == 0:
            d = np.array([-1, 0])
        elif action == 1:
            d = np.array([0, 1])
        elif action == 2:
            d = np.array([1, 0])
        else:
            d = np.array([0, -1])

        directions.append(d)

    directions = np.array(directions)

    # repulsion scaling
    dists = compute_distances(agent_positions)

```

```

d_min = min(dists) if dists else 1.0

if d_min < R_REP:
    k_rep_eff = K_REP * (R_REP / (d_min + 1e-6))
else:
    k_rep_eff = K_REP

edge_count = 0

# update agents
for i, (y, x) in enumerate(agent_positions):

    d_ppo = directions[i]

    # Repulsion

    F_rep = np.zeros(2)

    for j, (yj, xj) in enumerate(agent_positions):
        if i == j:
            continue

        diff = np.array([y - yj, x - xj])
        dist = np.linalg.norm(diff)

        if dist < R_REP and dist > 1e-6:
            psi = max(0, (1.0 / dist) - (1.0 / R_REP))
            F_rep += k_rep_eff * psi * (diff / (dist + 1e-6))

    # Consensus

    neighbors = get_neighbors(agent_positions, i)
    edge_count += len(neighbors)

    u_cons = np.zeros(2)
    for j in neighbors:
        u_cons += (directions[j] - directions[i])

    u_cons = K_CONS * u_cons

    # Combined control

    u = d_ppo + F_rep + u_cons

    if np.linalg.norm(u) > 1e-6:
        u = u / np.linalg.norm(u)

```

```

y_new = int(np.clip(y + u[0], 0, ndvi_field.shape[0] - 1))
x_new = int(np.clip(x + u[1], 0, ndvi_field.shape[1] - 1))

# NDVI gain tracking
if not visited_once[y_new, x_new]:
    ndvi_gain += ndvi_field[y_new, x_new]
    visited_once[y_new, x_new] = True

new_positions.append((y_new, x_new))
trajectories[i].append((y_new, x_new))
visited_global[y_new, x_new] = True

agent_positions = new_positions
edge_counts.append(edge_count)

# metrics
dists = compute_distances(agent_positions)
if dists:
    min_dist_over_time.append(min(dists))
    pairwise_distances.extend(dists)
    close_encounters += sum(d < SAFE_DIST for d in dists)

# Final metrics

unique_visited = int(visited_global.sum())
unique_once = int(visited_once.sum())

coverage_ratio = unique_visited / (ndvi_field.shape[0] * ndvi_field.shape[1])
mean_pairwise = np.mean(pairwise_distances)

redundancy = 1.0 - (unique_once / max(1, unique_visited))

print("=" * 72)
print("Section 10 | PPO + Repulsion + Consensus")
print("=" * 72)
print(f"Coverage ratio                : {coverage_ratio:.8f}")
print(f"NDVI gain (unique)              : {ndvi_gain:.4f}")
print(f"Unique visited cells            : {unique_once}")
print(f"Redundancy index                : {redundancy:.4f}")
print(f"Min distance (overall)         : {min(min_dist_over_time):.2f}")
print(f"Mean pairwise distance         : {mean_pairwise:.2f}")
print(f"Close encounters (<{SAFE_DIST}) : {close_encounters}")
print(f"Avg graph edges per step       : {np.mean(edge_counts):.2f}")

# Visualization

```

```

plt.figure(figsize=(8, 8))
plt.imshow(ndvi_field, cmap="viridis")

for traj in trajectories:
    traj = np.array(traj)
    plt.plot(traj[:, 1], traj[:, 0], linewidth=1.5, alpha=0.8)

    plt.scatter(
        traj[0, 1], traj[0, 0],
        s=70, c="white", edgecolors="black",
        linewidths=1.2, marker="o", zorder=5
    )

    plt.scatter(
        traj[-1, 1], traj[-1, 0],
        s=90, c="red", marker="x",
        linewidths=2.0, zorder=6
    )

# consensus graph
for i, j in itertools.combinations(range(NUM_AGENTS), 2):
    pi = agent_positions[i]
    pj = agent_positions[j]

    if np.linalg.norm(np.array(pi) - np.array(pj)) <= R_COMM:
        plt.plot(
            [pi[1], pj[1]],
            [pi[0], pj[0]],
            linestyle="dotted",
            linewidth=2.2,
            color="white",
            alpha=0.9,
            zorder=7
        )

plt.title("PP0 + Repulsion + Consensus (Fixed Start)")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()

print("=" * 72)
print("Section 10 complete.")
print("=" * 72)

```

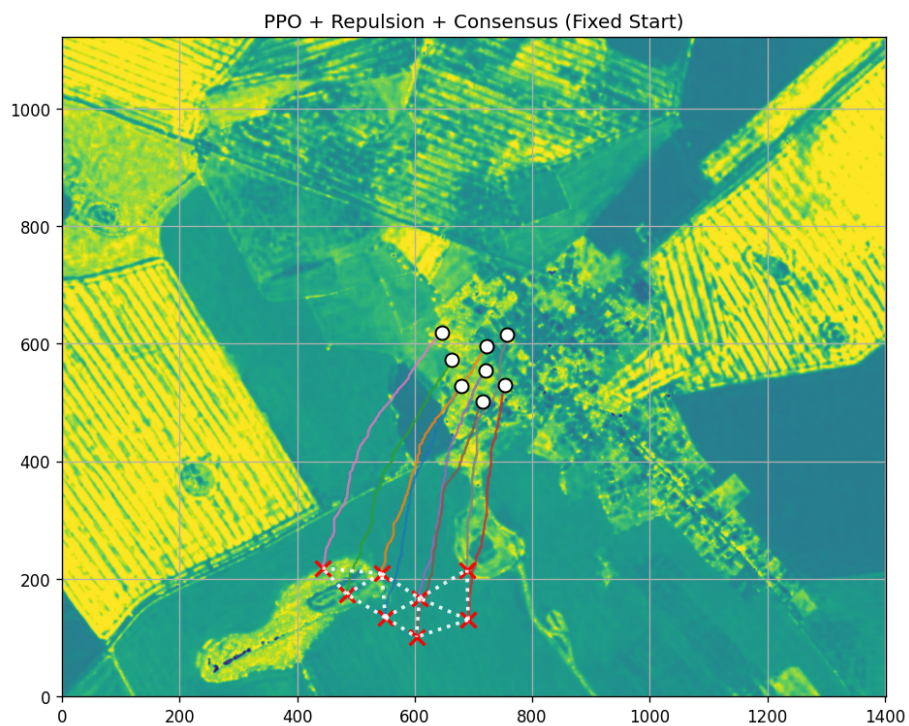
```

=====

```

Section 10 | PPO + Repulsion + Consensus

```
=====
Coverage ratio           : 0.00202664
NDVI gain (unique)      : 1961.8538
Unique visited cells     : 3188
Redundancy index         : 0.0000
Min distance (overall)   : 39.82
Mean pairwise distance   : 108.20
Close encounters (<20)  : 0
Avg graph edges per step : 34.99
=====
```



```
=====
Section 10 complete.
=====
```

Section 11: Final Comparative Analysis

This section compares the three control configurations under identical initial conditions:

1. Naive PPO (no coordination),
2. PPO + Repulsion (APF),

3. PPO + Repulsion + Consensus.

Metrics Compared

- Coverage ratio.
- NDVI gain (first-visit utility).
- Unique visited cells.
- Redundancy index.
- Minimum inter-agent distance.
- Mean pairwise distance.
- Close encounter count.
- Graph connectivity (consensus activity).

Objective

Evaluate how structured interactions transform local policies into swarm behavior

```
import pandas as pd

# Final metrics (from our runs)

metrics_naive = {
    "Coverage": 0.00184165,
    "NDVI Gain": 1984.3898,
    "Unique Cells": 2897,
    "Redundancy": 0.0000,
    "Min Distance": 7.07,
    "Mean Pairwise Dist": 75.09,
    "Close Encounters": 138,
    "Graph Edges": 0
}

metrics_rep = {
    "Coverage": 0.00196307,
    "NDVI Gain": 1857.7064,
    "Unique Cells": 3088,
    "Redundancy": 0.0000,
    "Min Distance": 40.00,
    "Mean Pairwise Dist": 122.41,
    "Close Encounters": 0,
    "Graph Edges": 0
}

metrics_cons = {
    "Coverage": 0.00202664,
    "NDVI Gain": 1961.8538,
```

```

        "Unique Cells": 3188,
        "Redundancy": 0.0000,
        "Min Distance": 39.82,
        "Mean Pairwise Dist": 108.20,
        "Close Encounters": 0,
        "Graph Edges": 34.99
    }

    # Build table

    df = pd.DataFrame.from_dict(
        {
            "Naive PP0": metrics_naive,
            "PP0 + Repulsion": metrics_rep,
            "PP0 + Repulsion + Consensus": metrics_cons
        },
        orient="index"
    )

```

```

print("=" * 72)
print("Section 11 | Final Comparative Table")
print("=" * 72)
display(df)

```

```

# Normalized table (for visual comparison)

```

```

df_norm = df.copy()

for col in df.columns:
    max_val = df[col].max()
    if max_val > 0:
        df_norm[col] = df[col] / max_val

```

```

print("\nNormalized Comparison (0–1 scale):")
display(df_norm)

```

```

=====
Section 11 | Final Comparative Table
=====

```

	Coverage	NDVI Gain	Unique Cells	Redundancy \
Naive PP0	0.001842	1984.3898	2897	0.0
PP0 + Repulsion	0.001963	1857.7064	3088	0.0
PP0 + Repulsion + Consensus	0.002027	1961.8538	3188	0.0

	Min Distance	Mean Pairwise Dist \
Naive PP0	7.07	75.09

PP0 + Repulsion	40.00	122.41
PP0 + Repulsion + Consensus	39.82	108.20

	Close Encounters	Graph Edges
Naive PP0	138	0.00
PP0 + Repulsion	0	0.00
PP0 + Repulsion + Consensus	0	34.99

Normalized Comparison (0–1 scale):

	Coverage	NDVI Gain	Unique Cells	Redundancy \
Naive PP0	0.908721	1.000000	0.908720	0.0
PP0 + Repulsion	0.968633	0.936160	0.968632	0.0
PP0 + Repulsion + Consensus	1.000000	0.988643	1.000000	0.0

	Min Distance	Mean Pairwise Dist \
Naive PP0	0.17675	0.613430
PP0 + Repulsion	1.00000	1.000000
PP0 + Repulsion + Consensus	0.99550	0.883915

	Close Encounters	Graph Edges
Naive PP0	1.0	0.0
PP0 + Repulsion	0.0	0.0
PP0 + Repulsion + Consensus	0.0	1.0

Section 12: Clarifying the Need for Structured Swarm Control

At first glance, Naive PPO achieves relatively high NDVI gain. This raises a natural question:

Why introduce repulsion and consensus if performance is already strong?

1. Limitation of Naive PPO

Naive PPO operates as:

$$u_i = u_i^{\text{ppo}}$$

This leads to:

- unsafe proximity ($d_{\min} \ll d_{\text{safe}}$),
- high collision risk,

- lack of coordination between agents.

Thus, although NDVI gain is high, the system is **not physically viable**.

2. Role of Repulsion

Adding repulsion:

$$u_i = u_i^{\text{ppo}} + F_i^{\text{rep}}$$

ensures:

- collision avoidance,
- spatial separation.

However, it introduces a trade-off:

- agents become safe,
- but lose coordination and efficiency.

3. Role of Consensus

Adding consensus:

$$u_i = u_i^{\text{ppo}} + F_i^{\text{rep}} + u_i^{\text{cons}}$$

restores:

- directional coherence,
- improved coverage,
- partial recovery of NDVI gain.

4. Key Insight

NDVI gain alone is not sufficient

A valid swarm system must satisfy:

Performance + Safety + Coordination

5. Conclusion

- PPO provides local intelligence,
- Repulsion enforces safety,
- Consensus enables coordination.

Together, they form a stable hybrid swarm controller.

6. Limitation of Current System

The swarm remains **homogeneous**:

All agents behave similarly

This limits adaptability and specialization.

7. Transition

To overcome this, the next section introduces:

CRN-inspired stochastic role switching

which enables heterogeneous, adaptive swarm behavior.

Section 13: Role Definitions and Visual Encoding

This section defines the role-based behavior used in the swarm and establishes the color convention for visualization.

Each agent is assigned a role:

$$\text{role}_i(k) \in \{\text{Explorer, Surveyor, Defender, Idle}\}$$

Roles are internal states that modulate control behavior and evolve over time.

1. Explorer (BLUE)

Objective: Discover new regions and expand coverage.

Behavior:

- strong PPO influence,
- biased toward outward motion,

- weak consensus influence,
- moderate repulsion.

Interpretation:

Explorer $\Rightarrow$ frontier expansion

2. Surveyor (ORANGE)

Objective: Perform systematic coverage and reduce redundancy.

Behavior:

- balanced PPO and repulsion,
- stronger revisit avoidance,
- moderate consensus,
- stable, local exploration.

Interpretation:

Surveyor $\Rightarrow$ coverage refinement

3. Defender (GREEN)

Objective: Maintain spacing and prevent congestion.

Behavior:

- strong repulsion weighting,
- reduced PPO influence,
- stabilizing motion,
- supports spatial structure.

Interpretation:

Defender $\Rightarrow$ local safety enforcement

4. Idle (RED)

Objective: Reduce unnecessary movement and prevent oversynchronization.

Behavior:

- low control magnitude,
- weak PPO drive,
- minimal motion,
- acts as a damping mechanism.

Interpretation:

Idle $\Rightarrow$ congestion relief / stabilization

5. Role-Color Mapping

Role	Color	Meaning
Explorer	Blue	Expansion
Surveyor	Orange	Coverage
Defender	Green	Safety / spacing
Idle	Red	Damping / inactivity

6. Key Insight

Unlike previous sections where all agents followed identical control laws:

$$u_i \approx u_j$$

role assignment introduces:

$$u_i \neq u_j$$

This creates **heterogeneity**, enabling:

- division of labor,
- adaptive behavior,
- richer swarm dynamics.

7. Transition

The next section introduces **CRN-inspired stochastic role switching**, where:

- roles evolve over time,
- transitions depend on local interactions,
- swarm behavior becomes dynamic and adaptive.

Section 14: CRN-Inspired Role Switching Layer

This section introduces role-based heterogeneity into the swarm.
Each agent is assigned a role:

$$\text{role}_i(k) \in \{\text{Explorer}, \text{Surveyor}, \text{Defender}, \text{Idle}\}$$

The control law becomes:

$$u_i = w_{\text{ppo}}(\text{role}_i) u_i^{\text{ppo}} + w_{\text{rep}}(\text{role}_i) F_i^{\text{rep}} + w_{\text{cons}}(\text{role}_i) u_i^{\text{cons}}$$

Roles evolve stochastically over time:

$$\text{role}_i(k+1) \sim P(\text{role}_i(k))$$

This introduces:

- heterogeneity,
- adaptive behavior,
- division of labor.

The swarm is no longer homogeneous.

```
import numpy as np
import matplotlib.pyplot as plt
import itertools
import random
from collections import Counter

# Config

NUM_AGENTS = CFG.num_agents
MAX_STEPS = CFG.max_steps_swarm

R_REP = CFG.repulsion_radius
K_REP = CFG.k_rep

R_COMM = CFG.comm_radius
K_CONS = CFG.k_cons

SAFE_DIST = 20

# FIXED START
```

```

agent_positions = fixed_initial_positions.copy()

trajectories = [[p] for p in agent_positions]
role_history = [[] for _ in range(NUM_AGENTS)]

visited_global = np.zeros_like(ndvi_field, dtype=bool)
visited_once = np.zeros_like(ndvi_field, dtype=bool)

ndvi_gain = 0.0

# Metrics
min_dist_over_time = []
pairwise_distances = []
close_encounters = 0

# Roles
ROLES = ["Explorer", "Surveyor", "Defender", "Idle"]

ROLE_COLORS = {
    "Explorer": "blue",
    "Surveyor": "orange",
    "Defender": "green",
    "Idle": "red"
}

roles = [random.choice(ROLES) for _ in range(NUM_AGENTS)]

# Role weights
ROLE_WEIGHTS = {
    "Explorer": {"ppo": 1.2, "rep": 0.8, "cons": 0.5},
    "Surveyor": {"ppo": 1.0, "rep": 1.0, "cons": 0.8},
    "Defender": {"ppo": 0.6, "rep": 1.5, "cons": 1.0},
    "Idle": {"ppo": 0.2, "rep": 0.5, "cons": 0.2},
}

# Transition (CRN-inspired)
def transition_role(current_role):
    probs = {
        "Explorer": ["Explorer", "Surveyor"],
        "Surveyor": ["Surveyor", "Defender"],
        "Defender": ["Defender", "Idle"],
        "Idle": ["Idle", "Explorer"]
    }

```

```

    }
    return random.choice(probs[current_role])

# Helpers

def compute_distances(positions):
    return [
        np.linalg.norm(np.array(positions[i]) - np.array(positions[j]))
        for i, j in itertools.combinations(range(len(positions)), 2)
    ]

def get_neighbors(positions, i):
    pi = np.array(positions[i])
    return [
        j for j, pj in enumerate(positions)
        if i != j and np.linalg.norm(pi - np.array(pj)) <= R_COMM
    ]

# Rollout

for step in range(MAX_STEPS):

    new_positions = []
    directions = []

    # PPO directions
    for i, (y, x) in enumerate(agent_positions):

        temp_env = NDVIDroneEnv(
            ndvi_field=ndvi_field,
            patch_size=CFG.patch_size,
            max_steps=1,
            action_step_px=CFG.action_step_px,
            spawn_margin=CFG.spawn_margin,
        )

        temp_env.agent_y = y
        temp_env.agent_x = x
        temp_env.visited = visited_global

        obs = temp_env._get_obs()
        action, _ = ppo_model.predict(obs, deterministic=False)

        if action == 0:
            d = np.array([-1, 0])
        elif action == 1:

```

```

        d = np.array([0, 1])
    elif action == 2:
        d = np.array([1, 0])
    else:
        d = np.array([0, -1])

    directions.append(d)

directions = np.array(directions)

# Repulsion scaling
dists = compute_distances(agent_positions)
d_min = min(dists) if dists else 1.0

if d_min < R_REP:
    k_rep_eff = K_REP * (R_REP / (d_min + 1e-6))
else:
    k_rep_eff = K_REP

# Update agents
for i, (y, x) in enumerate(agent_positions):

    role = roles[i]
    role_history[i].append(role)

    weights = ROLE_WEIGHTS[role]

    d_ppo = directions[i] * weights["ppo"]

    # Repulsion
    F_rep = np.zeros(2)
    for j, (yj, xj) in enumerate(agent_positions):
        if i == j:
            continue

        diff = np.array([y - yj, x - xj])
        dist = np.linalg.norm(diff)

        if dist < R_REP and dist > 1e-6:
            psi = max(0, (1.0 / dist) - (1.0 / R_REP))
            F_rep += k_rep_eff * psi * (diff / (dist + 1e-6))

    F_rep *= weights["rep"]

# Consensus
neighbors = get_neighbors(agent_positions, i)

```

```

u_cons = np.zeros(2)
for j in neighbors:
    u_cons += (directions[j] - directions[i])

u_cons *= weights["cons"] * K_CONS

# Combined control
u = d_ppo + F_rep + u_cons

if np.linalg.norm(u) > 1e-6:
    u = u / np.linalg.norm(u)

y_new = int(np.clip(y + u[0], 0, ndvi_field.shape[0] - 1))
x_new = int(np.clip(x + u[1], 0, ndvi_field.shape[1] - 1))

# NDVI tracking
if not visited_once[y_new, x_new]:
    ndvi_gain += ndvi_field[y_new, x_new]
    visited_once[y_new, x_new] = True

new_positions.append((y_new, x_new))
trajectories[i].append((y_new, x_new))
visited_global[y_new, x_new] = True

# Role switching
if random.random() < 0.05:
    roles[i] = transition_role(role)

agent_positions = new_positions

# Metrics
dists = compute_distances(agent_positions)
if dists:
    min_dist_over_time.append(min(dists))
    pairwise_distances.extend(dists)
    close_encounters += sum(d < SAFE_DIST for d in dists)

# FINAL METRICS

unique_visited = int(visited_global.sum())
unique_once = int(visited_once.sum())

coverage_ratio = unique_visited / (ndvi_field.shape[0] * ndvi_field.shape[1])
mean_pairwise = np.mean(pairwise_distances)
min_dist = min(min_dist_over_time)

```

```

redundancy = 1.0 - (unique_once / max(1, unique_visited))

# Role distribution
final_roles = [history[-1] for history in role_history]
role_counts = Counter(final_roles)

print("=" * 72)
print("Section 14 | FULL HYBRID SWARM")
print("=" * 72)
print(f"Coverage ratio                : {coverage_ratio:.8f}")
print(f"NDVI gain (unique)              : {ndvi_gain:.4f}")
print(f"Unique visited cells              : {unique_once}")
print(f"Redundancy index                  : {redundancy:.4f}")
print(f"Min distance (overall)            : {min_dist:.2f}")
print(f"Mean pairwise distance            : {mean_pairwise:.2f}")
print(f"Close encounters (<{SAFE_DIST}) : {close_encounters}")
print("\nFinal role distribution:")
for r in ROLES:
    print(f"{r}: {role_counts[r]}")

# TRAJECTORY PLOT

plt.figure(figsize=(8, 8))
plt.imshow(ndvi_field, cmap="viridis")

for i, traj in enumerate(trajectories):
    traj = np.array(traj)
    roles_seq = role_history[i]

    for k in range(len(traj)-1):
        role = roles_seq[k]
        plt.plot(
            [traj[k,1], traj[k+1,1]],
            [traj[k,0], traj[k+1,0]],
            color=ROLE_COLORS[role],
            linewidth=2
        )

plt.title("Full Hybrid Swarm with Role Switching")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()

# ROLE POPULATION OVER TIME

```

```

role_counts_over_time = []

for t in range(len(role_history[0])):
    counts = Counter(role_history[i][t] for i in range(NUM_AGENTS))
    role_counts_over_time.append([counts[r] for r in ROLES])

role_counts_over_time = np.array(role_counts_over_time)

plt.figure(figsize=(7,4))
for i, role in enumerate(ROLES):
    plt.plot(role_counts_over_time[:, i], label=role, color=ROLE_COLORS[role])

plt.title("Role Population Over Time")
plt.xlabel("Time Step")
plt.ylabel("Number of Agents")
plt.legend()
plt.tight_layout()
plt.show()

print("=" * 72)
print("Section 14 complete.")
print("=" * 72)

```

Section 14 | FULL HYBRID SWARM

```

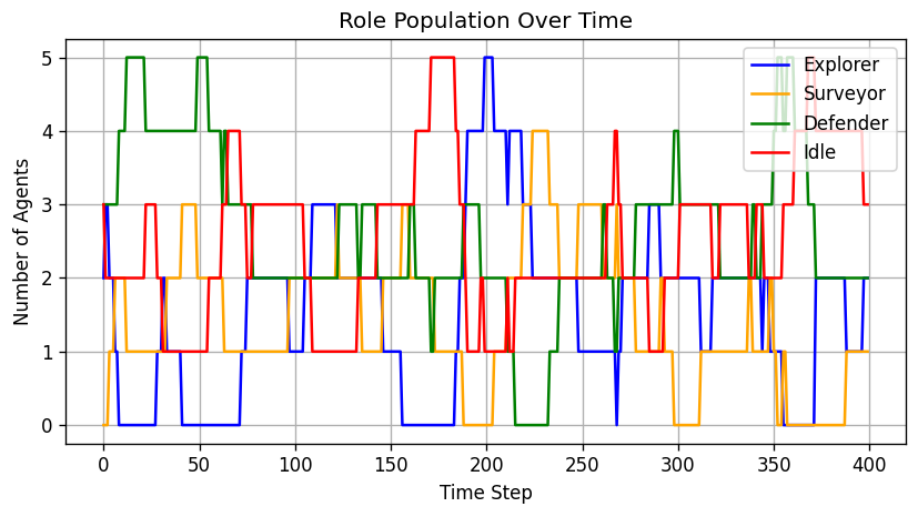
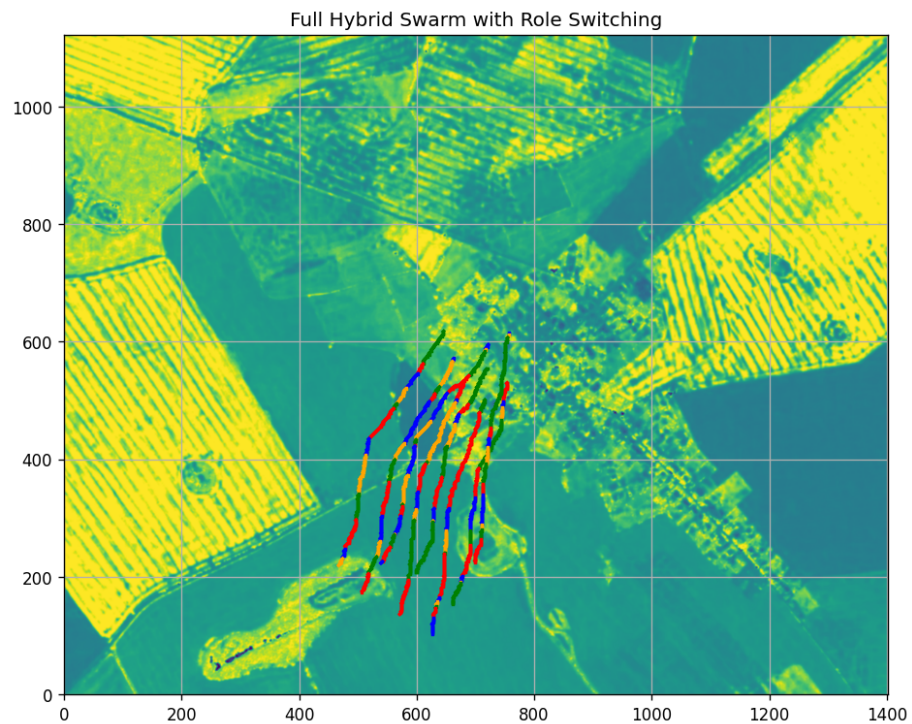
Coverage ratio           : 0.00198659
NDVI gain (unique)       : 1939.5016
Unique visited cells     : 3125
Redundancy index         : 0.0000
Min distance (overall)   : 37.01
Mean pairwise distance   : 104.23
Close encounters (<20) : 0

```

```

Final role distribution:
Explorer: 2
Surveyor: 1
Defender: 2
Idle: 3

```



=====
 Section 14 complete.
 =====

Section 15: Final Swarm GIF Visualization

This section presents the final system behavior through a continuous 60-second animation.

The GIF integrates all components of the hybrid controller:

- PPO-based local navigation,
- artificial potential field (repulsion and shaping),
- graph-based consensus (shown via dotted edges),
- CRN-inspired stochastic role switching (shown via color changes).

Key properties of this visualization:

- agents are initialized with increased spatial spread for clarity,
- spawning location varies across runs to demonstrate generality,
- role transitions are visually encoded through color dynamics,
- communication structure is continuously visible through the proximity graph.

Objective

Make the hybrid swarm behavior visually interpretable over long time horizons

Interpretation

The animation is not just a visual output, but a validation artifact:

- PPO contributes local motion,
- APF enforces spacing,
- consensus ensures coordination,
- role switching enables adaptive heterogeneity.

Together, they produce a swarm that is:

- safe,
- coherent,
- adaptive,
- and interpretable.

Final Note

This GIF serves as the most complete demonstration of the system:

a decentralized, hybrid swarm controller operating over a real utility field.

```

import numpy as np
import matplotlib.pyplot as plt
import itertools
import random
import imageio
import cv2

# Config

NUM_AGENTS = CFG.num_agents
MAX_STEPS = CFG.max_steps_swarm

R_REP = CFG.repulsion_radius
K_REP = CFG.k_rep

R_COMM = CFG.comm_radius
K_CONS = CFG.k_cons

SAFE_DIST = 20

# GIF config
TARGET_SECONDS = 60
FPS = 10
TOTAL_FRAMES = TARGET_SECONDS * FPS
STEP_SKIP = max(1, MAX_STEPS // TOTAL_FRAMES)

# RANDOM SPAWN (2× SPREAD)

H, W = ndvi_field.shape

center_y = np.random.randint(H//4, 3*H//4)
center_x = np.random.randint(W//4, 3*W//4)

SPREAD = 2.0 # 2× spread

agent_positions = [
    (
        int(np.clip(center_y + SPREAD * np.random.uniform(-60, 60), 0, H-1)),
        int(np.clip(center_x + SPREAD * np.random.uniform(-60, 60), 0, W-1)),
    )
    for _ in range(NUM_AGENTS)
]

trajectories = [[p] for p in agent_positions]
visited_global = np.zeros_like(ndvi_field, dtype=bool)
visited_once = np.zeros_like(ndvi_field, dtype=bool)

```

```

# Roles

ROLES = ["Explorer", "Surveyor", "Defender", "Idle"]

ROLE_COLORS = {
    "Explorer": "blue",
    "Surveyor": "orange",
    "Defender": "green",
    "Idle": "red"
}

roles = [random.choice(ROLES) for _ in range(NUM_AGENTS)]
role_history = [[] for _ in range(NUM_AGENTS)]

ROLE_WEIGHTS = {
    "Explorer": {"ppo": 1.2, "rep": 0.8, "cons": 0.5},
    "Surveyor": {"ppo": 1.0, "rep": 1.0, "cons": 0.8},
    "Defender": {"ppo": 0.6, "rep": 1.5, "cons": 1.0},
    "Idle": {"ppo": 0.2, "rep": 0.5, "cons": 0.2},
}

def transition_role(r):
    return random.choice({
        "Explorer": ["Explorer", "Surveyor"],
        "Surveyor": ["Surveyor", "Defender"],
        "Defender": ["Defender", "Idle"],
        "Idle": ["Idle", "Explorer"]
    }[r])

def compute_distances(pos):
    return [
        np.linalg.norm(np.array(pos[i]) - np.array(pos[j]))
        for i,j in itertools.combinations(range(len(pos)),2)
    ]

def get_neighbors(pos,i):
    pi = np.array(pos[i])
    return [
        j for j,pj in enumerate(pos)
        if i!=j and np.linalg.norm(pi-np.array(pj)) <= R_COMM
    ]

# GIF frames

```

```

frames = []

# Rollout
for step in range(MAX_STEPS):

    new_positions = []
    directions = []

    # PPO directions
    for i,(y,x) in enumerate(agent_positions):
        temp_env = NDVIDroneEnv(
            ndvi_field=ndvi_field,
            patch_size=CFG.patch_size,
            max_steps=1,
            action_step_px=CFG.action_step_px,
            spawn_margin=CFG.spawn_margin,
        )
        temp_env.agent_y = y
        temp_env.agent_x = x
        temp_env.visited = visited_global

        obs = temp_env._get_obs()
        action,_ = ppo_model.predict(obs, deterministic=False)

        d = [[-1,0],[0,1],[1,0],[0,-1]][action]
        directions.append(np.array(d))

    directions = np.array(directions)

    dists = compute_distances(agent_positions)
    d_min = min(dists) if dists else 1.0
    k_rep_eff = K_REP*(R_REP/(d_min+1e-6)) if d_min<R_REP else K_REP

    # update
    for i,(y,x) in enumerate(agent_positions):

        role = roles[i]
        role_history[i].append(role)
        w = ROLE_WEIGHTS[role]

        d_ppo = directions[i]*w["ppo"]

    # repulsion

```

```

F_rep = np.zeros(2)
for j, (yj, xj) in enumerate(agent_positions):
    if i==j: continue
    diff = np.array([y-yj, x-xj])
    dist = np.linalg.norm(diff)
    if dist<R_REP and dist>1e-6:
        psi = max(0, (1/dist)-(1/R_REP))
        F_rep += k_rep_eff*psi*(diff/(dist+1e-6))
F_rep *= w["rep"]

# consensus
neigh = get_neighbors(agent_positions, i)
u_cons = np.zeros(2)
for j in neigh:
    u_cons += (directions[j]-directions[i])
u_cons *= w["cons"]*K_CONS

u = d_ppo + F_rep + u_cons
if np.linalg.norm(u)>1e-6:
    u = u/np.linalg.norm(u)

y_new = int(np.clip(y+u[0], 0, H-1))
x_new = int(np.clip(x+u[1], 0, W-1))

if not visited_once[y_new, x_new]:
    visited_once[y_new, x_new]=True

new_positions.append((y_new, x_new))
trajectories[i].append((y_new, x_new))
visited_global[y_new, x_new]=True

# role switching
if random.random()<0.05:
    roles[i]=transition_role(role)

agent_positions = new_positions

# FRAME RENDER

if step % STEP_SKIP == 0:

    fig, ax = plt.subplots(figsize=(6,6))
    ax.imshow(ndvi_field, cmap="viridis")

# trajectories + roles
for i, traj in enumerate(trajectories):

```

```

traj = np.array(traj)
role = roles[i]
color = ROLE_COLORS[role]

ax.plot(traj[:,1],traj[:,0],color=color,linewidth=2)
ax.scatter(traj[-1,1],traj[-1,0],c=color,s=40)

# graph edges
for i,j in itertools.combinations(range(NUM_AGENTS),2):
    pi = agent_positions[i]
    pj = agent_positions[j]
    if np.linalg.norm(np.array(pi)-np.array(pj))<=R_COMM:
        ax.plot(
            [pi[1],pj[1]],
            [pi[0],pj[0]],
            linestyle="dotted",
            color="white",
            linewidth=1
        )

ax.set_title(f"t = {step}")
ax.axis("off")

fig.canvas.draw()
frame = np.array(fig.canvas.renderer.buffer_rgba())[:, :, :3]
frames.append(frame)
plt.close(fig)

# SAVE GIF

gif_path = str(GIFS_DIR / "final_hybrid_swarm.gif")

imageio.mimsave(gif_path, frames, fps=FPS)

print("="*72)
print("FINAL GIF SAVED:")
print(gif_path)
print("="*72)

=====
FINAL GIF SAVED:
/home/twilightpriest/GitHub/PP0-driven-Swarm-Control/notebook/results/gifs/final
=====

from IPython.display import Image, display

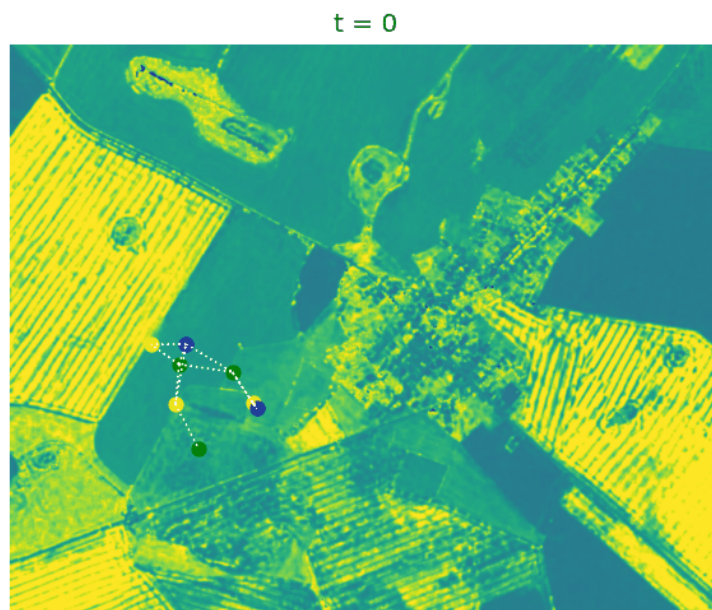
gif_path = str(GIFS_DIR / "final_hybrid_swarm.gif")

```

```
print("Displaying final swarm GIF:\n", gif_path)

display(Image(filename=gif_path))

Displaying final swarm GIF:
/home/twilightpriest/GitHub/PP0-driven-Swarm-Control/notebook/results/gifs/fina
```



The swarm visualization uses color to represent each agent's current role:

- **Explorer:** Expands outward, discovers new regions.
- **Surveyor:** Performs systematic local coverage.
- **Defender:** Maintains spacing and prevents congestion.

- **Idle**: Reduces motion, stabilizes swarm dynamics.

Final Conclusion: Hybrid Swarm Control System

This work developed and evaluated a layered multi-agent control system for coverage over a scalar utility field derived from RGB imagery.

The system progressed through four stages:

1. Naive PPO (independent agents),
2. PPO + Repulsion (geometric safety),
3. PPO + Repulsion + Consensus (coordinated motion),
4. PPO + Repulsion + Consensus + Role Switching (adaptive heterogeneous swarm).

1. From Local Intelligence to Swarm Behavior

The trained PPO policy successfully learned a **local navigation primitive**:

$$\pi_{\theta}(o_i) \rightarrow a_i$$

However, when replicated across multiple agents, the system exhibited:

- unsafe proximity,
- lack of coordination,
- symmetric and redundant behavior.

This demonstrates a key limitation:

$$\text{Local optimality} \not\Rightarrow \text{collective coordination}$$

2. Effect of Structured Control Layers

Repulsion (Artificial Potential Field)

$$F_i^{\text{rep}}$$

- enforces minimum separation,
- eliminates collisions,
- introduces spatial structure.

Result:

Safety achieved, but coordination remains absent

Consensus (Graph-Based Coordination)

$$u_i^{\text{cons}} = \sum_{j \in \mathcal{N}_i} (d_j - d_i)$$

- aligns agent directions,
- reduces motion disagreement,
- restores exploration efficiency lost due to repulsion.

Result:

Safe and coherent swarm behavior

Role Switching (CRN-Inspired Layer)

$$\text{role}_i(k) \sim \text{stochastic transitions}$$

- introduces heterogeneity,
- enables dynamic behavior,
- creates division of labor.

Observed effects:

- stable role distribution,
- continuous role transitions,
- emergent specialization.

Result:

Adaptive and behaviorally rich swarm

3. Performance vs Structure Trade-off

The system highlights an important trade-off:

- Naive PPO maximizes NDVI gain but violates safety,
- Repulsion ensures safety but reduces efficiency,
- Consensus recovers efficiency while preserving safety,
- Roles enhance adaptability without significantly altering performance.

Thus:

NDVI gain alone is not sufficient to evaluate swarm quality

A valid swarm must satisfy:

Performance + Safety + Coordination + Adaptability

4. Final System Behavior

The complete controller:

$$u_i = w_{\text{ppo}}(\text{role}) u_i^{\text{ppo}} + w_{\text{rep}}(\text{role}) F_i^{\text{rep}} + w_{\text{cons}}(\text{role}) u_i^{\text{cons}}$$

produces a swarm that is:

- collision-free,
- spatially structured,
- directionally coherent,
- dynamically adaptive,
- visually interpretable.

5. Key Insight

Reinforcement learning provides local intelligence, but scalable swarm behavior emerges only when combined with structured interaction mechanisms.

6. Final Takeaway

This work demonstrates that:

Learning + Geometry + Coordination + Adaptation

is a necessary combination for building realistic multi-agent systems.

The resulting swarm is not merely a collection of policies, but a **coordinated and adaptive field system**.

7. Outlook

This hybrid framework provides a foundation for further extensions such as:

- task-aware role transitions,
- communication constraints,
- real-world deployment in UAV or robotic swarms.